

QT

Aladine Chetouani

Polytech'Orléans – site Galilée, Orléans

F 311

Objectifs

- Types de programmes:
 - Console
 - Fait précédemment
 - GUI (Graphical User Interface)
 - Interface graphique
 - On doit utiliser des bibliothèques spécifiques (Winform, Qt, ...)

- Bibliothèques multiplateforme

- Windows

- Linux

- Mac OS

- ...

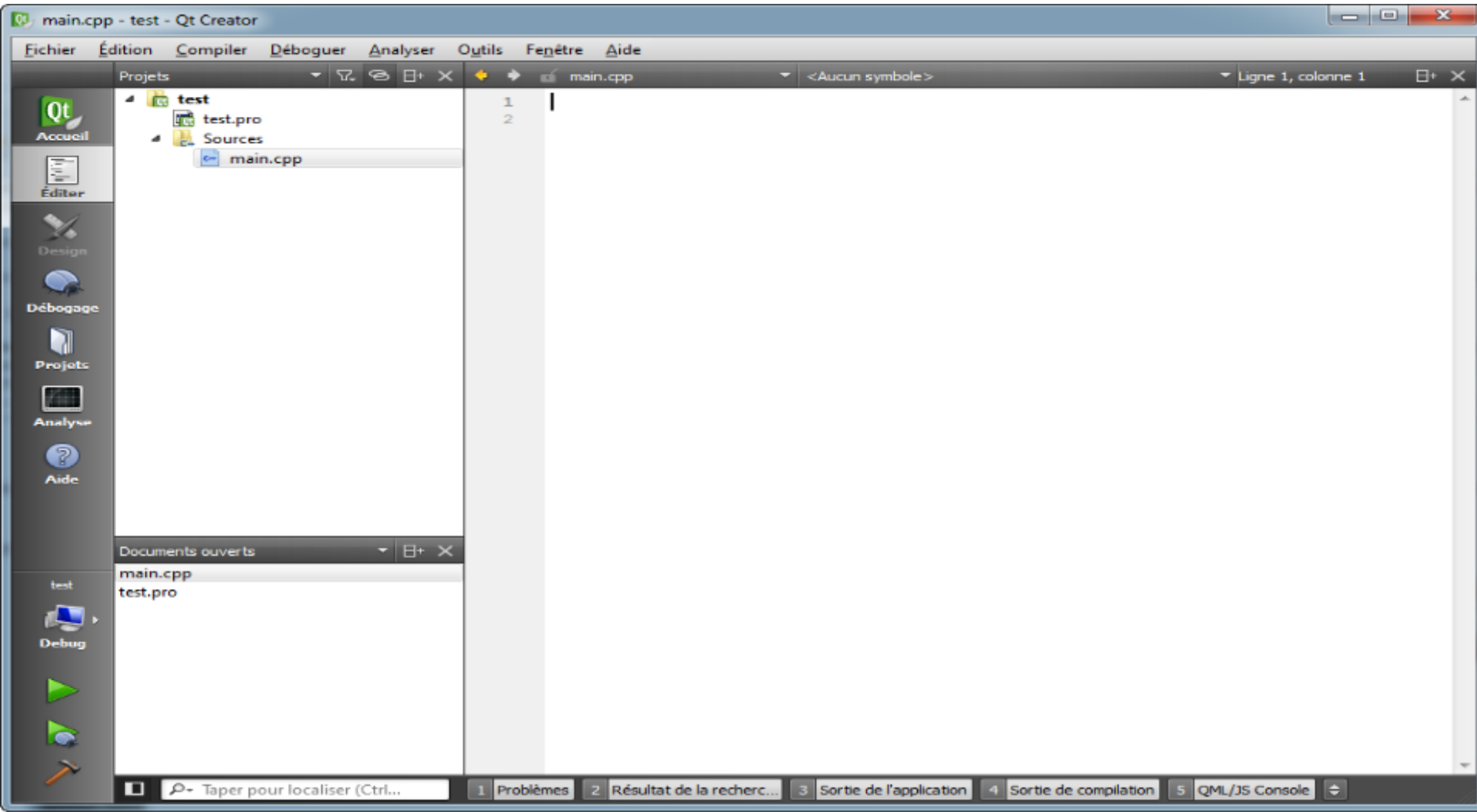
- Portage avec une simple recompilation (sur les systèmes les plus classiques : Linux, Mac, Windows, ...)



- Composer de plusieurs modules:
 - GUI
 - OpenGL (3D)
 - Dessin
 - Réseau
 - XML
 - SQL (MySQL, Oracle, ...)
 - ...

QT Creator

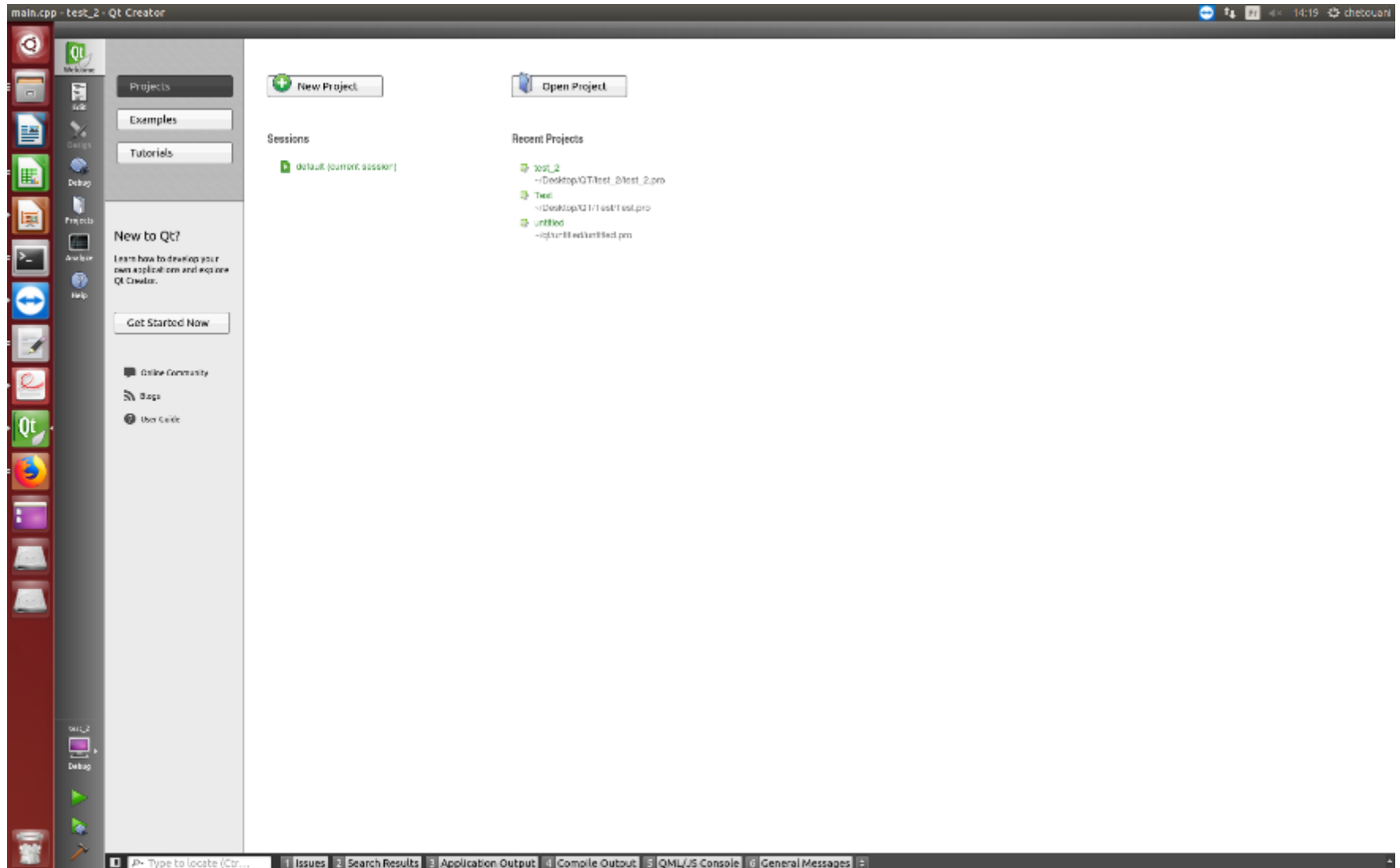
- Interface



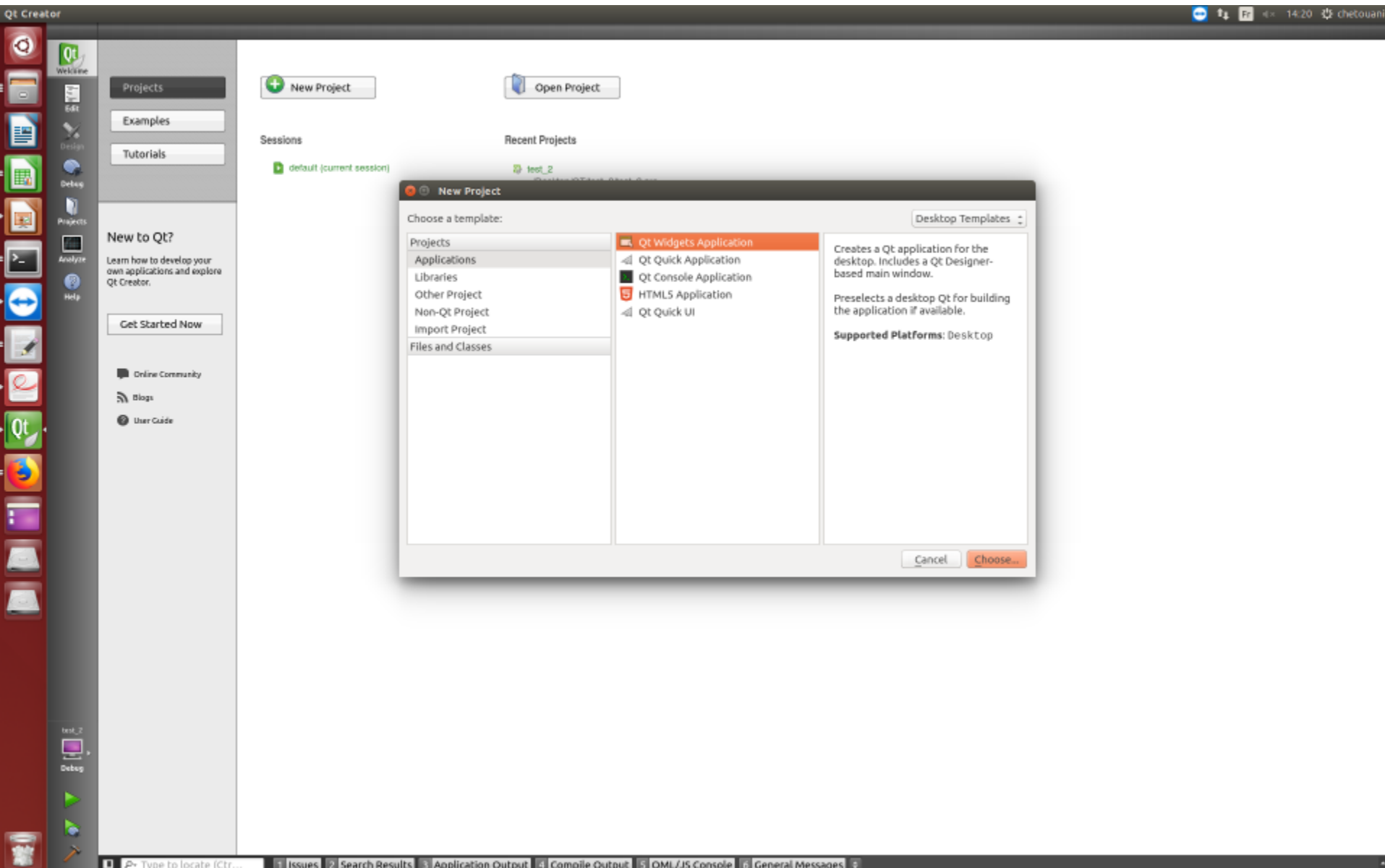
Première application : code

New Project → Applications → Qt Widgets
Application

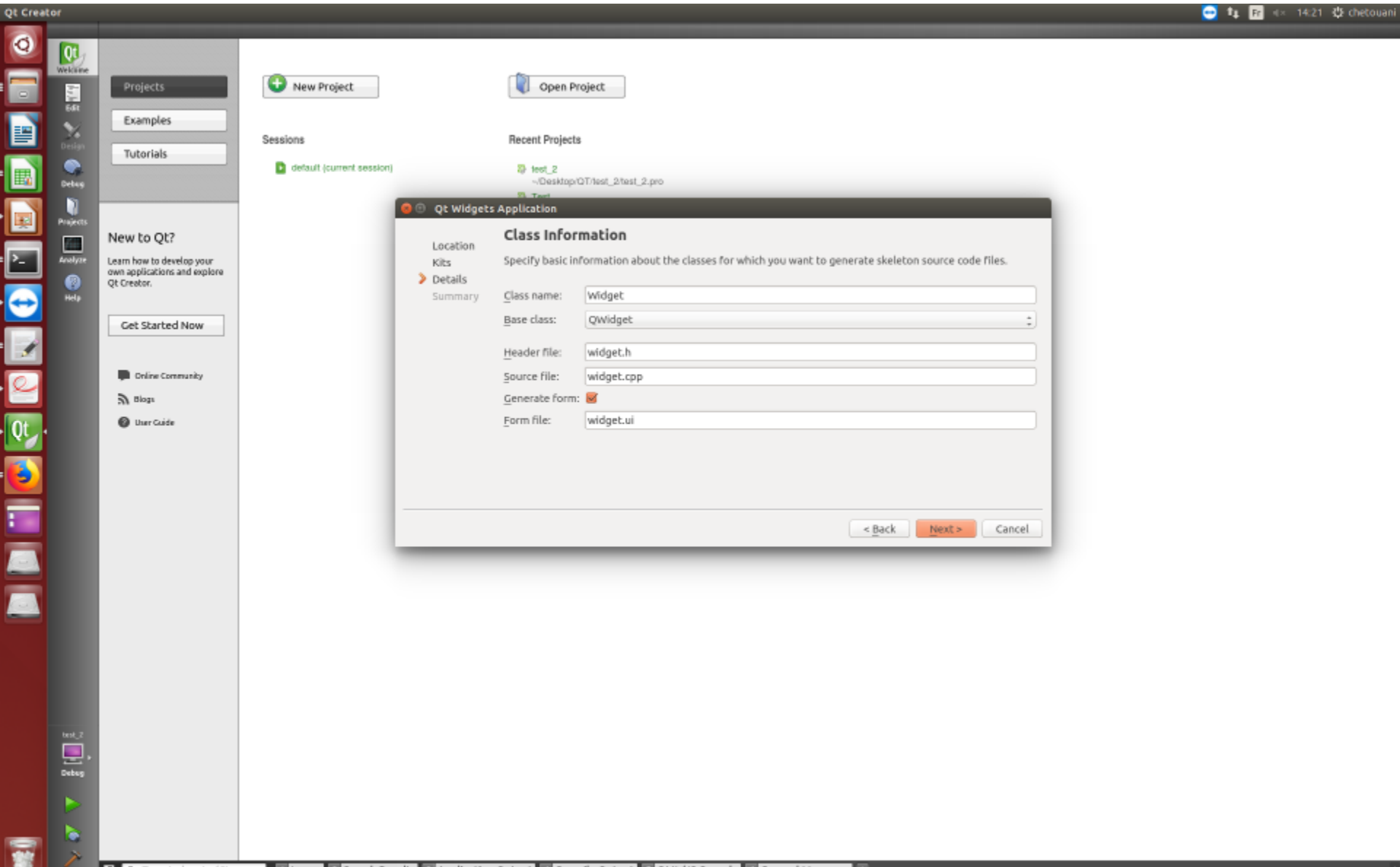
Première application : code



Première application : code



Première application : code



Première application : code (minimal)

```
#include <QApplication>
```

```
int main(int argc, char *argv[])  
{  
    QApplication app(argc, argv);  
    return app.exec();  
}
```

Première application : code

```
#include <QApplication>
```

- Inclure cette classe
 - Plus besoin de la bibliothèque iostream
- Chaque application qui contient une IHM QT doit avoir une instance de cette classe

Première application : code

```
int main(int argc, char *argv[])
```

- Fonction principale
- Rappel : Point d'entrée du programme
- argc ?
- argv ?

Première application : code

`QApplication app(argc, argv);`

- Instance de la classe QApplication
- Créer avant d'utiliser les fonctionnalités d'IHM QT

Première application : code

```
return app.exec();
```

- La fonction `main()` passe la « main » à Qt
- `exec()`
 - Démarre notre programme
 - Affichage des fenêtres
- `return`
 - Fin du programme

Aller plus loin : code

```
#include <QApplication>
#include <QPushButton>

int main(int argc, char *argv[]) {

    QApplication app(argc, argv);
    QPushButton Hello("Hello World");
    Hello.show();
    return app.exec();

}
```

Aller plus loin : code

`#include <QPushButton>`

- Ajouter les bibliothèques liées aux objets utilisés
 - Ici, bouton d'interface graphique
- En général, pour chaque QWidget, il y a une classe à inclure

Aller plus loin : code

```
QPushButton hello("Hello world!");
```

- Liée à l'IHM
- Constructeur
 - Plusieurs constructeurs disponibles (documentation)
- Création d'un bouton qui affiche le texte « Hello World! »

Aller plus loin : code

```
hello.show();
```

- Permet de rendre visible le bouton
 - Modale : fenêtre principale (on ne peut pas utiliser les autres fenêtres)

Aller plus loin : code

- Pour exécution
 Modifier le fichier .pro
- Ajouter QT += widgets
- Exécuter **qmake** (pour la compilation)
 clique droit sur le projet, « run qmake »
 (principe du makefile)
- Diffuser le programme:
 besoin de fichiers DLL

Aller plus loin : code

- Utilitaire **qmake**

fichier projet (*.pro)

description des fichiers qui composent le programme (fichiers sources, librairies, ...)

génération

des fichiers makefile sous Linux

des projets Visual Studio sous Windows

...

Aller plus loin : code

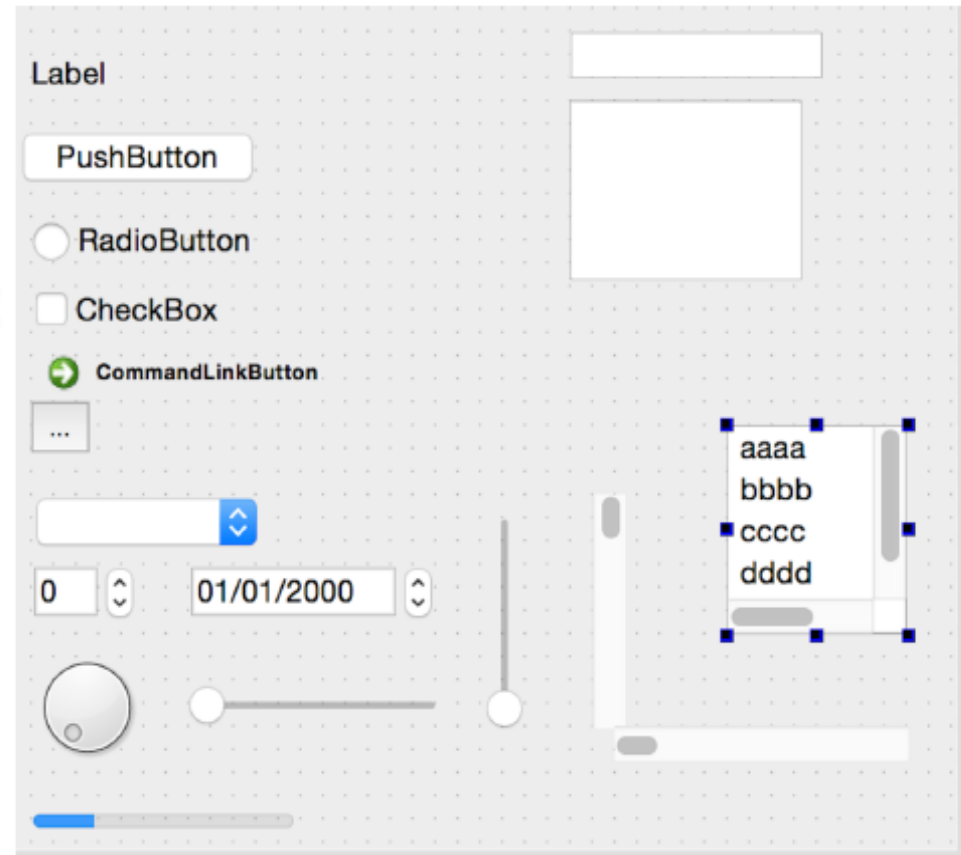
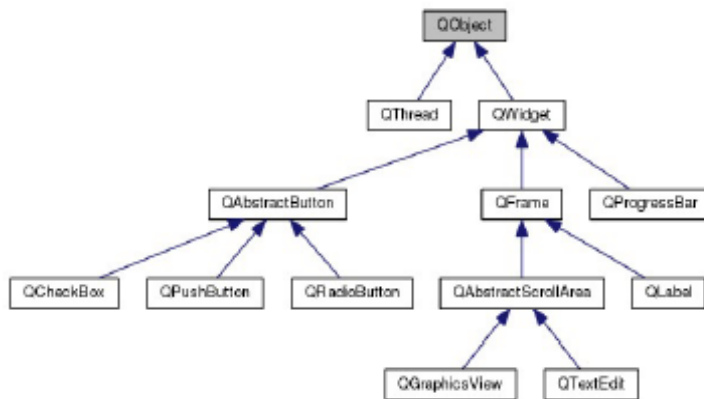
- Utilitaire **qmake**

balises

SOURCES	fichiers *.cpp
HEADERS	fichiers *.h
TEMPLATE	type sortie
TARGET	nom de la sortie
CONFIG	extensions à utiliser
LIBS	bibliothèques ajoutées
INCLUDEPATH	chemin vers en-têtes des bibliothèques utilisées

Les widgets les plus courants

Interacteurs



Les widgets les plus courants

- Pour concevoir une interface graphique, il faut connaître les widgets courants de la librairie graphique utilisée :
 - **QLabel**
 - Texte
 - **QcheckBox**
 - Boîte à cocher
 - **QLCDNumber**
 - Afficheur digital

Les widgets les plus courants

- Pour concevoir une interface graphique, il faut connaître les widgets courants de la librairie graphique utilisée :
 - **QLineEdit**
 - Une ligne
 - **QTextEdit**
 - multi-ligne
 - **QListView**
 - Liste d'éléments

Les widgets les plus courants

- Pour concevoir une interface graphique, il faut connaître les widgets courants de la librairie graphique utilisée :
 - **QMenu**
 - Menus (normaux, pop up)
 - **QProgressBar**
 - Barre de progression
 - **QPushButton**
 - Bouton

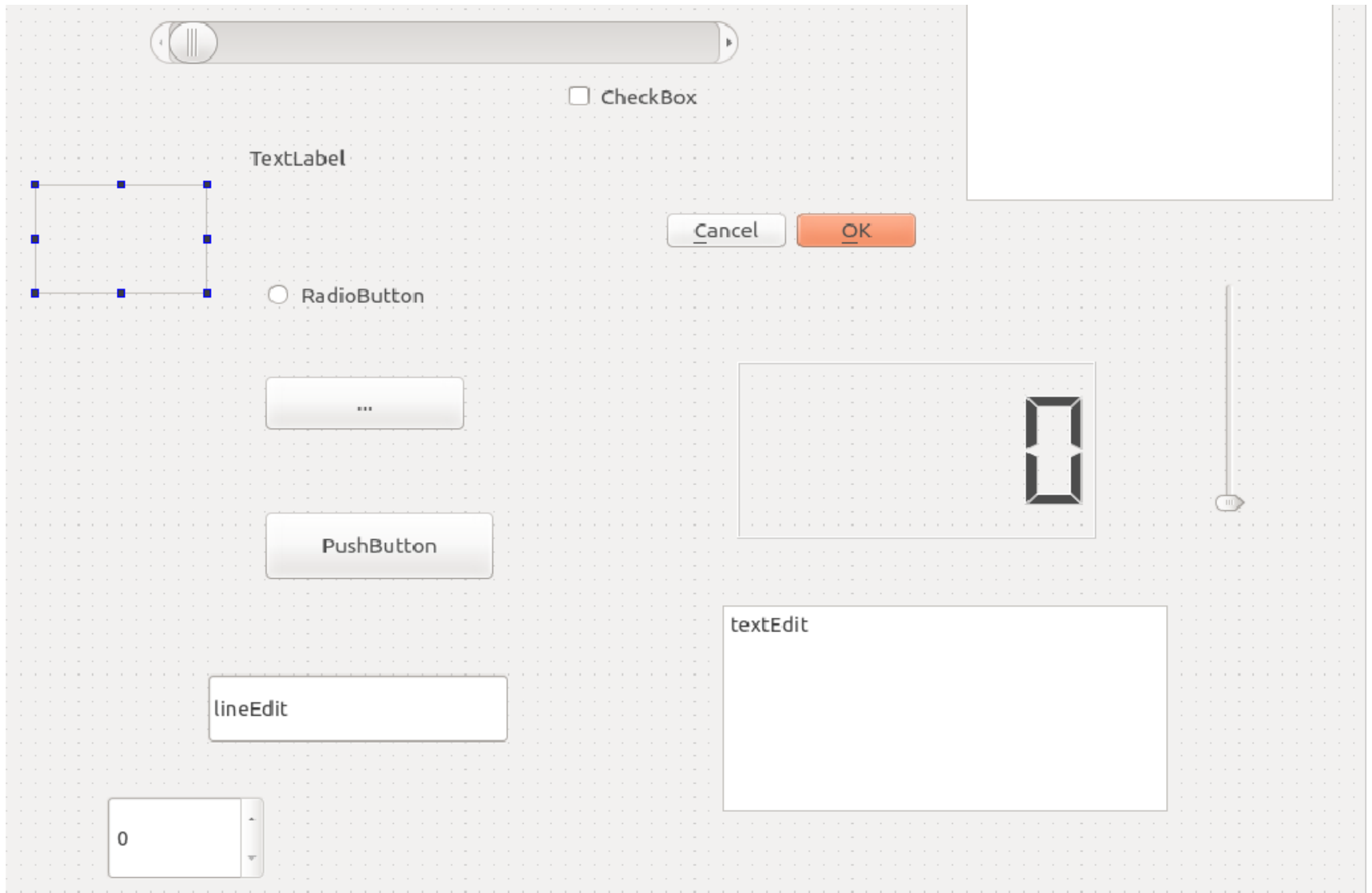
Les widgets les plus courants

- Pour concevoir une interface graphique, il faut connaître les widgets courants de la librairie graphique utilisée :
 - **QRadioButton**
 - Bouton radio
 - **QScrollBar**
 - Barre de défilement (horizontale ou verticale)
 - **QSlider**
 - Barre de niveau (horizontale ou verticale)

Les widgets les plus courants

- Pour concevoir une interface graphique, il faut connaître les widgets courants de la librairie graphique utilisée :
 - **QSpinBox**
 - Boîte de saisie avec des flèches haut/bas
 - **QToolButton**
 - Bouton pour les barres d'outils
 - **QWhatsThis**
 - Fenêtre d'informations sur les widgets

Les widgets les plus courants : exemple



Les widgets les plus courants

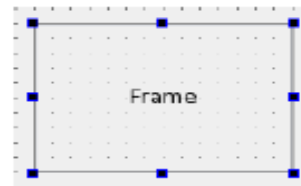
- Widget permettant de regrouper des widgets
 - **QFrame**
 - Classe de base pour les widgets avec frames
 - **QGroupBox**
 - Groupe horizontalement ou verticalement les éléments dans un cadre (ligne)
 - **QHBoxLayout**
 - Groupe horizontalement les éléments sans frame
 - **QVBoxLayout**
 - Groupe verticalement les éléments sans frame

Les widgets les plus courants : exemple

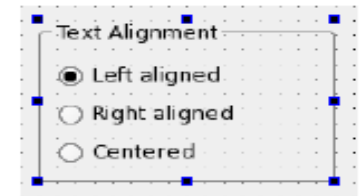
Conteneurs

QWidget

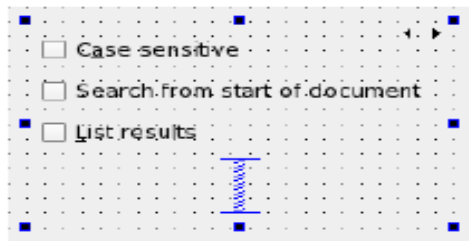
QDockWidget



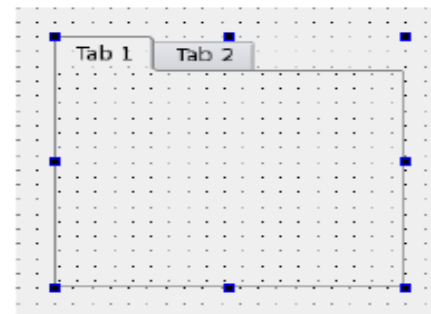
QFrame



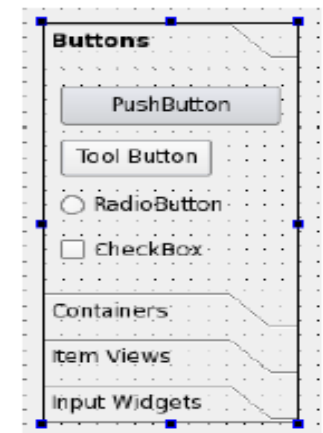
QGroupBox



QStackedWidget



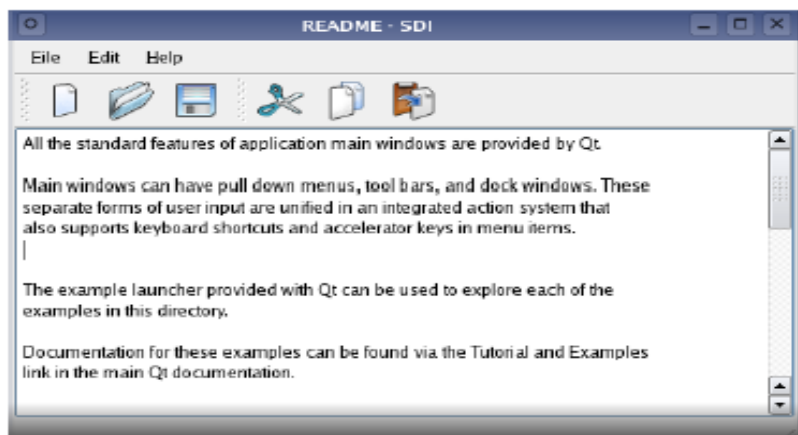
QTabWidget



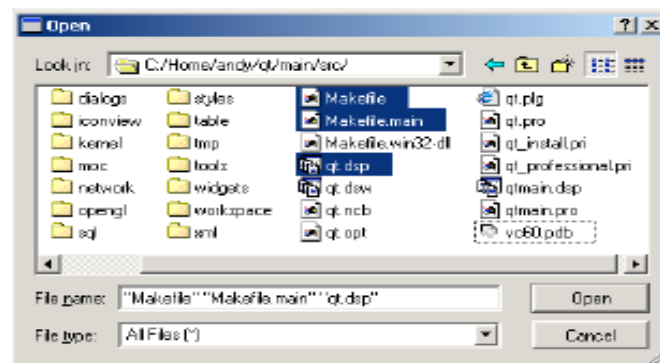
QToolbox

Les widgets les plus courants : exemple

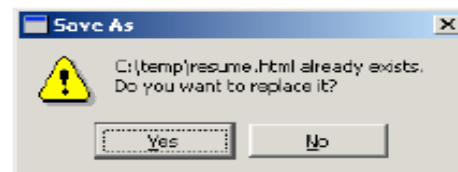
Fenêtre principale, boîtes de dialogue, menus



QMainWindow



QFileDialog



QMessageBox

Les Widgets : propriétés

- Utilisation des fonctionnalités disponibles
 - Chaque Widget à ses propres propriétés (héritage)
 - Accessibles via les accesseurs
 - pour la lecture (get)
 - pour l'écriture (set)

Les Widgets : propriétés

Exemples :

- **Définir la taille**

```
hello.resize(75, 30);
```

- **Définir la font**

```
hello.setFont(QFont("Times", 18, QFont::Bold));
```

il faut ajouter `#include <QFont>` et créer un objet `QFont`

- **Pour un bouton**

```
QPushButton Hello("Hello World");
```

```
Hello.setText("Bonjour");
```

```
Hello.setTooltip("Help");
```

```
QFont myFont("Times New Roman");
```

```
Hello.setFont(myFont);
```

ou

```
Hello.setFont(QFont("Times New Roman"));
```

Les Widgets : propriétés

Exemples :

setCursor

lorsque la souris est sur le bouton
plusieurs possibilités

```
Hello.setCursor(Qt::PointingHandCursor);
```



plusieurs types de curseur disponibles

Qt Licensing

All Qt Overviews

Constant	Value	Description
Qt::ArrowCursor	0	 The standard arrow cursor.
Qt::UpArrowCursor	1	 An arrow pointing upwards toward the top of the screen.
Qt::CrossCursor	2	 A crosshair cursor, typically used to help the user accurately select a point on the screen.
Qt::WaitCursor	3	 An hourglass or watch cursor, usually shown during operations that prevent the user from interacting with the application.
Qt::IBeamCursor	4	 A caret or ibeam cursor, indicating that a widget can accept and display text input.
Qt::SizeVerCursor	5	 A cursor used for elements that are used to vertically resize top-level windows.
Qt::SizeHorCursor	6	 A cursor used for elements that are used to horizontally resize top-level windows.
Qt::SizeBDiagCursor	7	 A cursor used for elements that are used to diagonally resize top-level windows at their top-right and bottom-left corners.
Qt::SizeFDiagCursor	8	 A cursor used for elements that are used to diagonally resize top-level windows at their top-left and bottom-right corners.
Qt::SizeAllCursor	9	 A cursor used for elements that are used to resize top-level windows in any direction.
Qt::BlankCursor	10	A blank/invisible cursor, typically used when the cursor shape needs to be

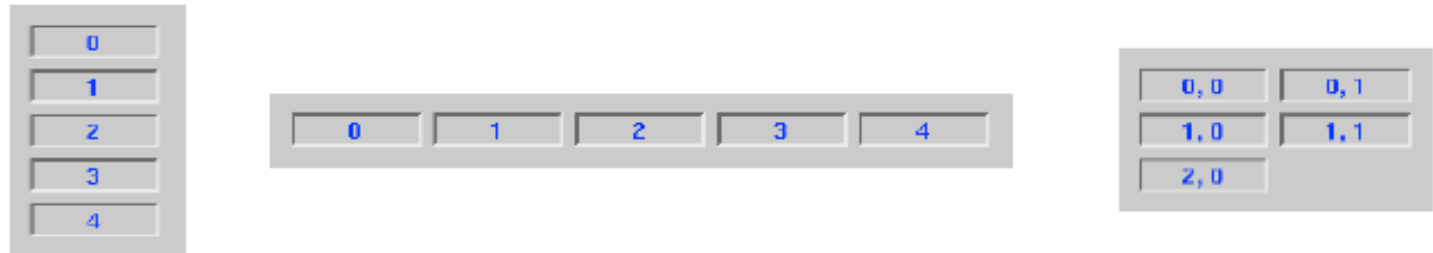
Gestionnaire de placement

« Layout Manager »

- Positionnement des Widgets
- **absolu (fixe)**
 - problème lors du redimensionnement (interdire le redimensionnement)
 - résolution
- **relative**
 - agencement en fonction des autres Widgets

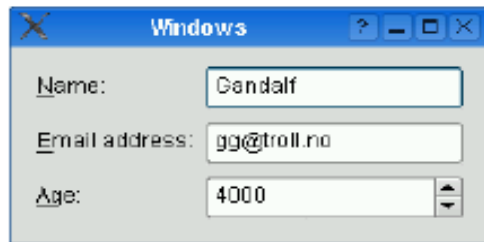
Gestionnaire de placement

« Layout Manager »

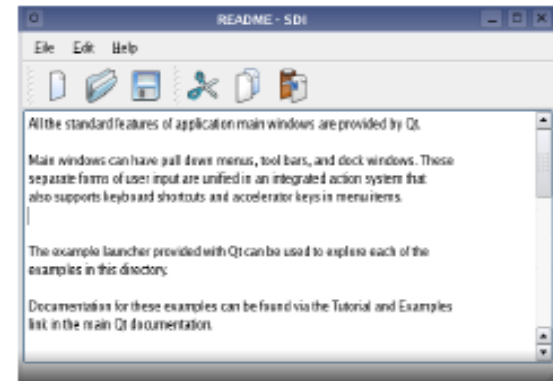


QHBoxLayout, QVBoxLayout, QGridLayout

Notion commune à plusieurs outils IHM : Winform, Android, ...



QFormLayout



QMainWindow

Gestionnaire de placement

« Layout Manager »

- Héritage de la classe abstraite QLayout
- Placement initial
- Redimensionnement de la fenêtre
- Deux types :
- **QBoxLayout** : Horizontalement ou verticalement
- **QGridLayout** : Sous forme d'un tableau

Gestionnaire de placement

« Layout Manager »

QBoxLayout

Les widgets sont rangés selon l'ordre dans lequel ils sont ajoutés au layout

Exemples : Plusieurs boutons

Placement directement dans la fenêtre

```
QPushButton *b1 = new QPushButton("1", &windows);  
QPushButton *b2 = new QPushButton("2", &windows);  
QPushButton *b3 = new QPushButton("3", &windows);
```

Placement dans un layout

```
QPushButton *b1 = new QPushButton("1");  
QPushButton *b2 = new QPushButton("2");  
QPushButton *b3 = new QPushButton("3");
```

```
QHBoxLayout *layout = new QHBoxLayout;  
layout->addWidget(b1);  
layout->addWidget(b2);  
layout->addWidget(b3);
```

```
// c'est le layout qui est placé dans la fenêtre  
fenetre.setLayout(layout);
```

Gestionnaire de placement

« Layout Manager »

```
#include <QApplication>
#include <QPushButton>
#include <QHBoxLayout>
```

```
int main(int argc, char *argv[]) {
```

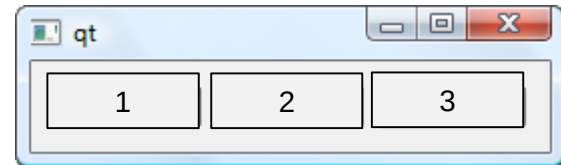
```
    QApplication app(argc, argv);
    QWidget fenetre;
    QPushButton *b1 = new QPushButton("1");
    QPushButton *b2 = new QPushButton("2");
    QPushButton *b3 = new QPushButton("3");
```

```
    QHBoxLayout *layout = new QHBoxLayout;
    layout->addWidget(b1);
    layout->addWidget(b2);
    layout->addWidget(b3);
```

```
    fenetre.setLayout(layout);
```

```
    fenetre.show();
    return app.exec();
```

```
}
```



Gestionnaire de placement

« Layout Manager »

```
#include <QApplication>
#include <QPushButton>
#include <QVBoxLayout>
```

```
int main(int argc, char *argv[]) {
```

```
    QApplication app(argc, argv);
    QWidget fenetre;
    QPushButton *b1 = new QPushButton("1");
    QPushButton *b2 = new QPushButton("2");
    QPushButton *b3 = new QPushButton("3");
```

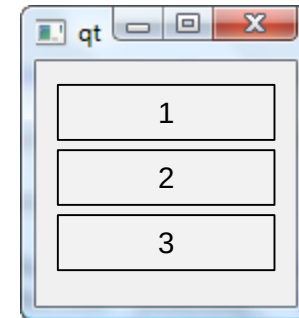
```
    QVBoxLayout *layout = new QVBoxLayout;
```

```
    layout->addWidget(b1);
    layout->addWidget(b2);
    layout->addWidget(b3);
```

```
    fenetre.setLayout(layout);
```

```
    fenetre.show();
    return app.exec();
```

```
}
```



Gestionnaire de placement

« Layout Manager »

QGridLayout

un tableau dont chaque cellule est identifiable par son numéro de ligne et de colonne (débutant à 0)

Possibilités

- de définir la structure du tableau

- un widget sur plusieurs cases

- décaler les Widgets

Gestionnaire de placement

« Layout Manager »

```
#include <QApplication>
#include <QPushButton>
#include <QHBoxLayout>
```

```
int main(int argc, char *argv[]) {
```

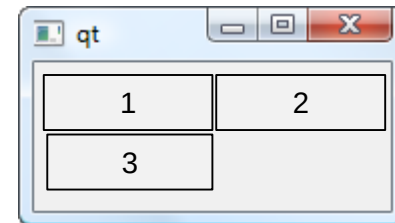
```
    QApplication app(argc, argv);
    QWidget fenetre;
    QPushButton *b1 = new QPushButton("1");
    QPushButton *b2 = new QPushButton("2");
    QPushButton *b3 = new QPushButton("3");
```

```
    QGridLayout *layout = new QGridLayout;
    layout->addWidget(b1, 0, 0);
    layout->addWidget(b2, 0, 1);
    layout->addWidget(b3, 1, 0);
```

```
    fenetre.setLayout(layout);
```

```
    fenetre.show();
    return app.exec();
```

```
}
```



Gestionnaire de placement « Layout Manager »



Gestionnaire de placement

« Layout Manager »

Peuvent être imbriqués

```
QVBoxLayout * v_layout = new QVBoxLayout( );  
v_layout->addWidget( new QPushButton( "OK" ) );  
v_layout->addWidget( new QPushButton( "Cancel" ) );  
v_layout->addStretch( );  
v_layout->addWidget( new QPushButton( "Help" ) );
```

```
QListBox * country_list = new QListBox( this );  
countryList->insertItem( "Canada" );  
...etc...
```

```
QHBoxLayout * h_layout = new QHBoxLayout( );  
h_layout->addWidget( country_list );  
h_layout->addLayout( v_layout );
```

Gestionnaire de placement

« Layout Manager »

QFormLayout

pour les formulaires (possibilités d'utiliser des QLabel et QTextEdit)

Simple d'utilisation

- création du Widget QFormLayout

- ajout des éléments (QLineEdit, QTextEdit, ...)

Gestionnaire de placement

« Layout Manager »

QFormLayout

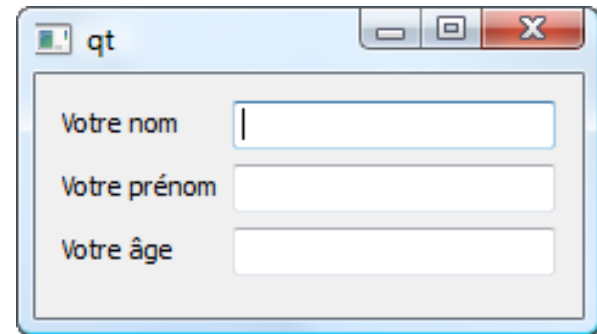
Exemple

```
#include <QApplication>
#include <QLineEdit>
#include <QFormLayout>
int main(int argc, char *argv[]) {
    QApplication app(argc, argv);
    QWidget fenetre;

    QLineEdit *nom = new QLineEdit;
    QLineEdit *prenom = new QLineEdit;
    QLineEdit *age = new QLineEdit;

    QFormLayout *layout = new QFormLayout;
    layout->addRow("Votre nom", nom);
    layout->addRow("Votre prénom", prenom);
    layout->addRow("Votre âge", age);

    fenetre.setLayout(layout);
    fenetre.show();
    return app.exec();
}
```



Personnaliser son Widget

Création d'une nouvelle classe qui hérite de la classe QWidget (MyFirstClass.h et MyFirstClass.cpp)

```
#ifndef DEF_MYFIRSTAPPLICATION  
#define DEF_MYFIRSTAPPLICATION
```

```
#include <QApplication>  
#include <QWidget>  
#include <QPushButton>
```

```
class MyFirstClass : public QWidget {  
    public:  
        MyFirstClass();  
  
    private:  
        QPushButton *m_bouton;  
};  
  
#endif
```

Pour éviter les problèmes liés aux « include » plusieurs fois

Héritage (accès aux attributs [si existants] et propriétés)

→ #include<QWidget>

Personnaliser son Widget

Création d'une nouvelle classe qui hérite de la classe QWidget
(MyFirstClass.h et MyFirstClass.cpp)

Appel du constructeur

```
#include "MyFirstClass.h"
```

```
MyFirstClass::MyFirstClass () : QWidget(){
```

```
    m_bouton = new QPushButton("Bouton test", this);
```

```
    m_bouton->setFont(QFont("Comic Sans MS", 14));
```

```
    m_bouton->setCursor(Qt::PointingHandCursor);
```

```
}
```

**second paramètre correspond au Widget parent
ici c'est le Widget lui-même (this)**

Personnaliser son Widget

Création d'une nouvelle classe qui hérite de la classe QWidget
(MyFirstClass.h et MyFirstClass.cpp)

```
#include <QApplication>
#include "MyFirstClass.h"

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    MyFirstClass myFClass;
    myFClass.show();

    return app.exec();
}
```

Classe QDialog

- Modale ou non modale
 - Non modale :
 - indépendante des autres fenêtres
 - Méthode show()
 - Modale :
 - Bloque les autres fenêtres
 - Exec() ou setModal(true) et show()
- Valeur de retour
- Bouton ou pas

Classe Qdialog

- Boîtes de dialogue disponibles
 - QDialog
 - QColorDialog
 - QFontDialog
 - QFileDialog
 - QMessageBox
 - QMessageBox::critical()
 - QMessageBox::information()
 - QMessageBox::question(),
 - QMessageBox::warning()
 - QErrorMessage

Classe QDialog

- QMessageBox

- Plusieurs methods disponibles
 - information : permettant d'afficher les informations saisies avec une icône information

StandardButton information (QWidget * **parent**, const QString & **title**, const QString & **text**, StandardButtons **buttons = Ok**, StandardButton **defaultButton= NoButton**);

Exemple

```
QMessageBox::information(this, "Titre", "Test");
```

Classe QDialog

- QMessageBox
 - Possibilité de mettre en forme le texte avec du code HTML

Exemple

```
QMessageBox::information(this, "Titre", "<strong>Test HTML</strong>");
```

Classe QDialog

- QMessageBox
 - Même principe avec
 - QMessageBox::warning
 - QMessageBox::critical
 - QMessageBox::question
 - ...

Classe QDialog

- QMessageBox
 - Possibilité de personnaliser les boutons
 - 4ème argument

```
QMessageBox::question(this, "Titre", "Test réussi", QMessageBox::Yes |  
                    QMessageBox::No);
```

Classe QDialog

- QMessageBox
 - Possibilité de personnaliser les boutons
 - 4ème argument
 - Comment récupérer la réponse ?

```
int retour=QMessageBox::question(this, "Titre", "Test réussi", QMessageBox::Yes |  
QMessageBox::No);  
if (retour== QMessageBox::Yes){  
    ....  
}  
else if (retour == QMessageBox::No){  
    ...  
}
```

Classe QDialog

- **QInputDialog**
 - Possibilité de saisir
 - du texte
 - un entier
 - un decimal
 - sélectionner un élément dans une liste

Classe QDialog

- **QInputDialog**

- Possibilité de saisir
 - du texte
 - un entier
 - un decimal
 - sélectionner un élément dans une liste

Classe QDialog

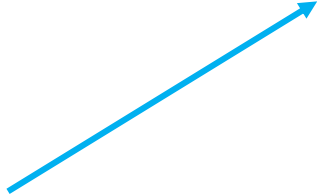
- **QInputDialog**

Mode : affichage ou non des caractères (mot de passe)

QString QInputDialog::getText (QWidget * parent, const QString & title, const QString & label, **QLineEdit::EchoMode mode = QLineEdit::Normal**, **const QString & text = QString()**, bool * ok = 0, Qt::WindowFlags f = 0);

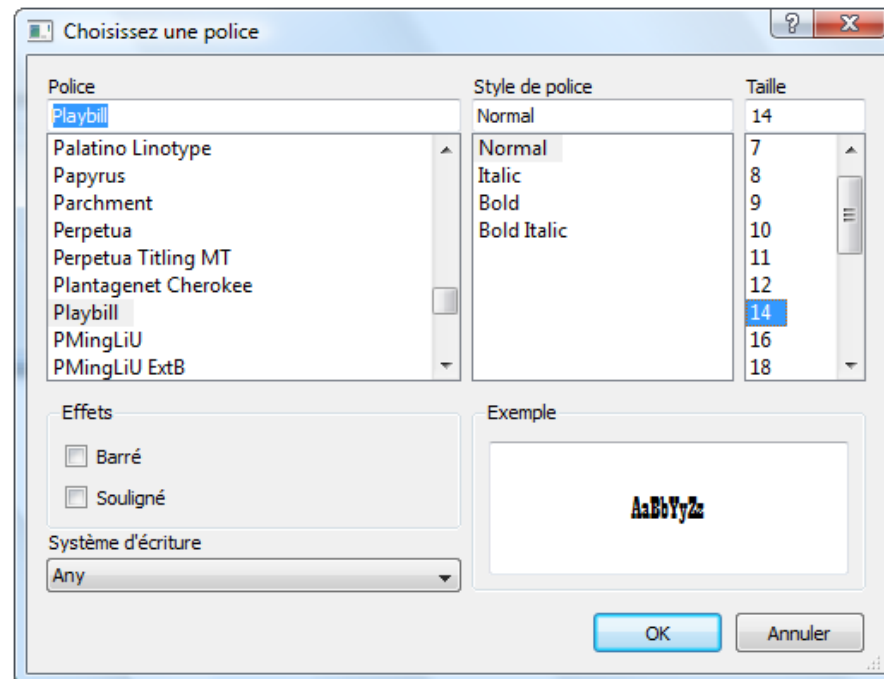


Texte par défaut



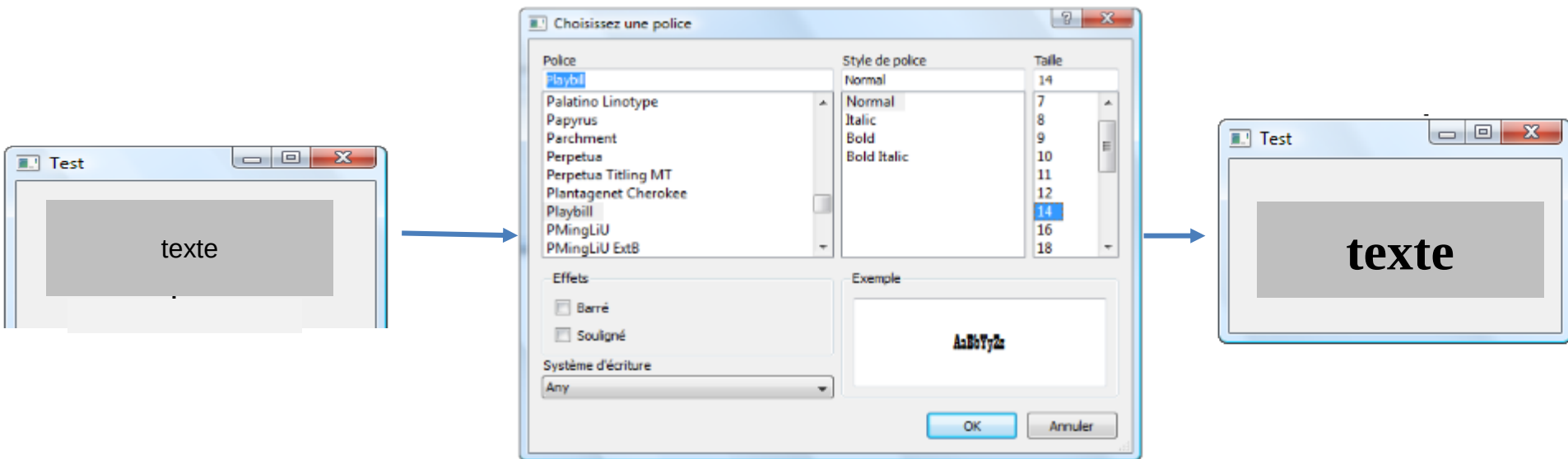
Classe QDialog

- **QFontDialog**
 - Donne la possibilité à l'utilisateur de modifier les caractéristiques de la police



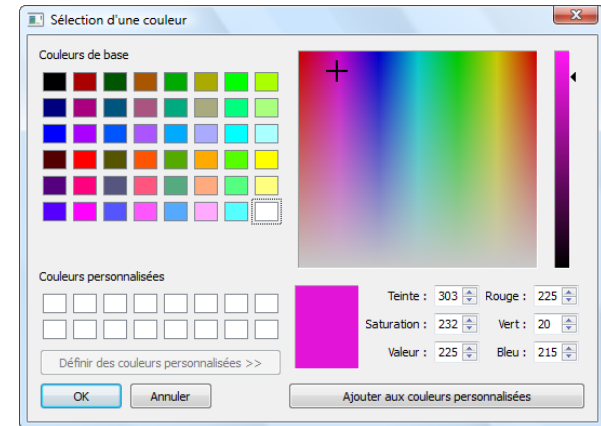
Classe QDialog

- **QFontDialog**
 - exemple



Classe QDialog

- **Plusieurs autres fenêtres de dialogue**
 - QColorDialog
 - QFileDialog
 - getOpenFileName
 - getSaveFileName

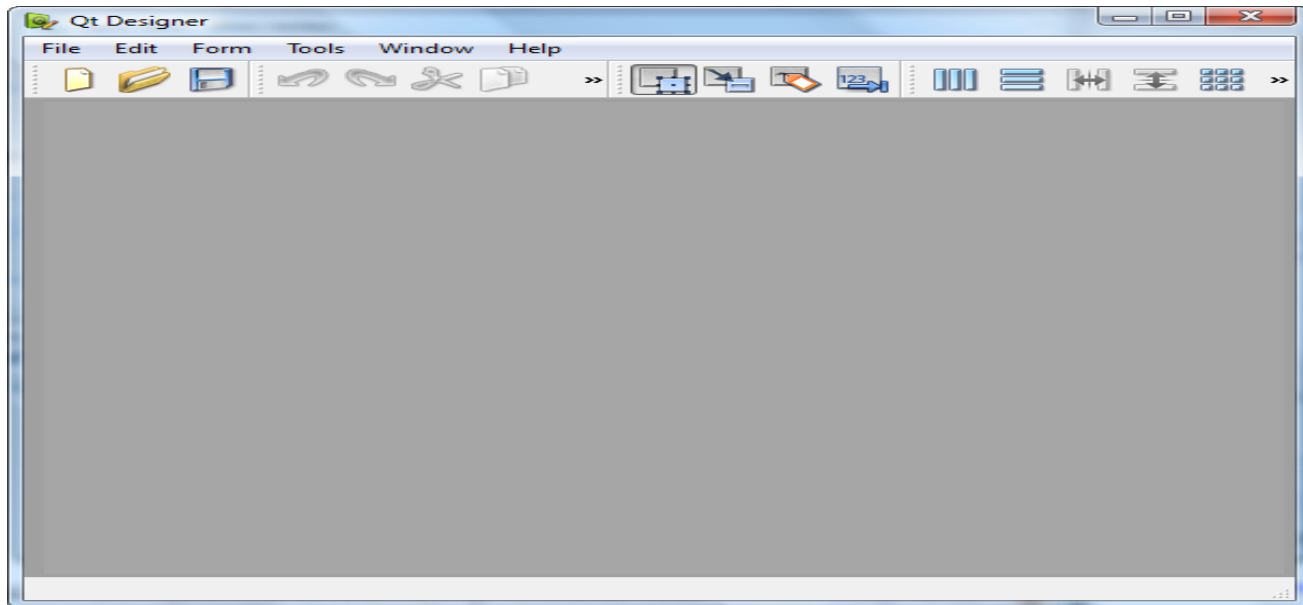


```
QString fichier = QFileDialog::getOpenFileName(this, "Ouvrir ", QString(), "Texte (*.txt)");
```

```
QString fichier = QFileDialog::getSaveFileName(this, "Enregistrer", QString(), "Texte (*.txt)");
```

QMainWindow

- Permet de mettre en place des interfaces avec des menus, barre d'outils, ...
- Hérite de QWidget



QMainWindow

- Exemple

```
#include <QtWidgets>

class MyFirstClass: public QMainWindow{
    public:
        MyFirstClass();

    private:

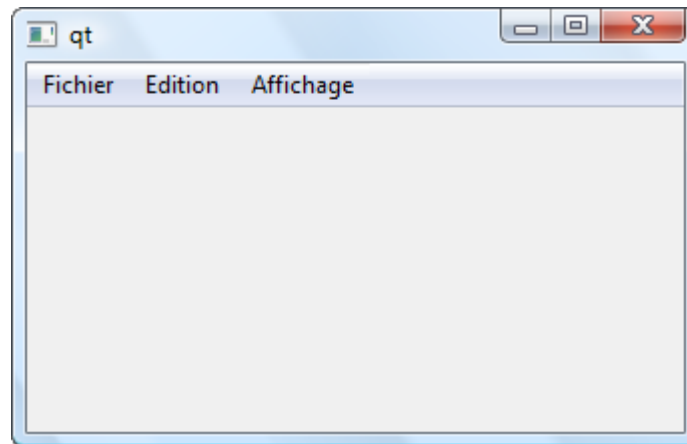
};
```

QMainWindow

- **Définition des menus**

& : permet de définir des raccourcis (à placer avant la lettre choisie)

```
QMenu *mFichier = menuBar()->addMenu("&Fichier");  
QMenu *mEdition = menuBar()->addMenu("&Edition");  
QMenu *mAffichage = menuBar()->addMenu("&Affichage");
```



QMainWindow

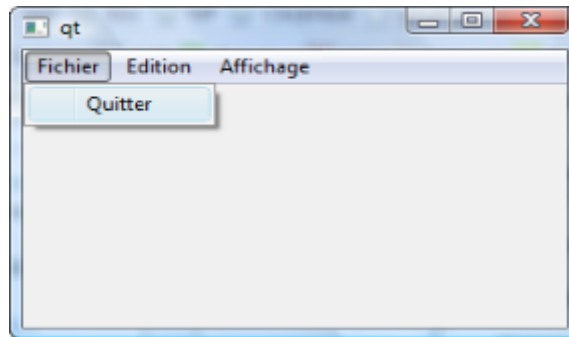
- **Définition des menus**

```
QMenu *mFichier = menuBar()->addMenu("&Fichier");  
QMenu *mEdition = menuBar()->addMenu("&Edition");  
QMenu *mAffichage = menuBar()->addMenu("&Affichage");
```

& : permet de définir des raccourcis (à placer avant la lettre choisie)

QMainWindow

- **Définition des menus**
 - Association d'actions pour rendre le menu actif



```
QMenu *mFichier = menuBar()->addMenu("&Fichier");  
QAction *actionQuitter = new QAction("&Quitter", this);  
mFichier->addAction(actionQuitter);
```

Création de l'action

Ajout de l'action au menu

QMainWindow

- **Définition des menus**
 - Rendre actif l'action
 - Connexion signal/slot

```
QMenu *mFichier = menuBar()->addMenu("&Fichier");  
QAction *actionQuitter = new QAction("&Quitter", this);  
mFichier->addAction(actionQuitter);
```

```
connect(actionQuitter, SIGNAL(triggered()), qApp, SLOT(quit()));
```

QMainWindow

- **Définition des menus**
 - Possibilité
 - d'ajouter des raccourcis clavier
 - `actionQuitter->setShortcut(QKeySequence("Ctrl+Q"));`
 - d'ajouter des icônes
 - `actionQuitter->setIcon(QIcon("quitter.png"));`
 - ...

QMainWindow

- **Gestion de la barre de tâches**
 - Utilisation d'actions
 - Possibilité d'ajouter des Widgets
 - Séparateur
 - ...

QMainWindow

- Deux types de QMainWindow (zone centrale)
 - SDI
 - Single Document Interface (affichage d'un document à la fois)
 - MDI
 - Multiple Document Interface (possibilité d'afficher plusieurs documents à la fois)

QMainWindow

- SDI
 - Insertion dans la zone centrale d'un Widget

```
QWidget *zoneCentrale = new QWidget;  
setCentralWidget(zoneCentrale);
```

Dans le constructeur

QMainWindow

Dans le constructeur

- SDI: possibilité d'ajouter des Widgets

```
FenPrincipale::FenPrincipale(){  
    QWidget *zoneCentrale = new QWidget;  
  
    QLineEdit *nom = new QLineEdit;  
    QLineEdit *prenom = new QLineEdit;  
    QLineEdit *age = new QLineEdit;  
  
    QFormLayout *layout = new QFormLayout;  
    layout->addRow("Votre nom", nom);  
    layout->addRow("Votre prénom", prenom);  
    layout->addRow("Votre âge", age);  
  
    zoneCentrale->setLayout(layout);  
  
    setCentralWidget(zoneCentrale);  
}
```

QMainWindow

Dans le constructeur

- MDI: ajout de sous fenêtres dans la zone centrale

QMdiArea: conteneur de widgets

```
QMdiArea *zoneCentrale = new QMdiArea;
```

```
QTextEdit *zoneTexte1 = new QTextEdit;
```

```
QTextEdit *zoneTexte2 = new QTextEdit;
```

```
QMdiSubWindow *sousFenetre1 = zoneCentrale->addSubWindow(zoneTexte1);
```

```
QMdiSubWindow *sousFenetre2 = zoneCentrale->addSubWindow(zoneTexte2);
```

```
setCentralWidget(zoneCentrale);
```

QMdiSubWindow : sous fenêtre qui va afficher un widget à l'intérieur. Possibilité d'ajouter un Qui va lui-même contenir plusieurs Widget (layout)

QMainWindow

- MDI: ajout de sous fenêtres dans la zone centrale

```
QMdiArea *zoneCentrale = new QMdiArea;
```

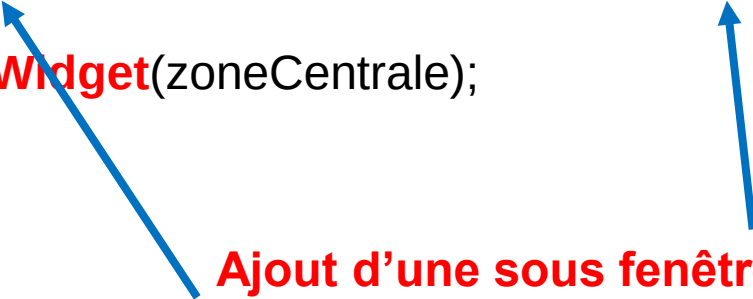
```
QTextEdit *zoneTexte1 = new QTextEdit;
```

```
QTextEdit *zoneTexte2 = new QTextEdit;
```

```
QMdiSubWindow *sousFenetre1 = zoneCentrale->addSubWindow(zoneTexte1);
```

```
QMdiSubWindow *sousFenetre2 = zoneCentrale->addSubWindow(zoneTexte2);
```

```
setCentralWidget(zoneCentrale);
```



**Ajout d'une sous fenêtre dans la zone centrale
Mais retourne un pointeur QMdiSubWindow ?**

QMainWindow

- MDI: ajout de sous fenêtres dans la zone centrale
- Possibilité d'affecter un titre à chaque fenêtre
 - `setWindowTitle()`
- Possibilité d'ajouter une icône
 - `setWindowIcon()`

Gestion des événements

Comment gérer les événements ?
clique bouton, souris, ...

QT → Signaux / Slots

Signal/Slot

Signal

envoi d'un signal par un Widget quand il change d'état (ex: bouton clique)

pas d'indication de destination les signaux sont connectés à un ou des slots

Slot

récepteur (méthode)

appelé automatiquement dès que le signal connecté est émis

Signal/Slot

Différentes possibilités:

Un signal ➔ 1 ou plusieurs slots (un événement peut engendrer plusieurs appels)

Plusieurs signaux ➔ 1 slot

L'émetteur du signal

ne connaît pas le ou les récepteurs (slot(s))

n'a pas d'information d'acquiescement (signal reçu ou non)

Le récepteur ne connaît pas l'émetteur

Signal/Slot

- Établissement de la connexion signal/slot
utilisation de la méthode **QObject::connect()**

4 arguments:

- 1/ pointeur vers l'émetteur
- 2/ nom du signal
- 3/ pointeur vers le récepteur
- 4/ Slot à exécuter lors de la réception du signal

clique Slot appelé lors de l'événement

↓ ↓

QObject::connect(b1, SIGNAL(clicked()), App, SLOT(quit()));

↑ ↑

bouton application

The diagram illustrates the four arguments of the `QObject::connect()` function. Red arrows point from descriptive labels to the corresponding parts of the code: `b1` is labeled 'bouton' (button), `SIGNAL(clicked())` is labeled 'clique' (click), `App` is labeled 'application', and `SLOT(quit())` is labeled 'Slot appelé lors de l'événement' (slot called during the event).

Signal/Slot

Exemple

```
#include <QApplication>
#include <QPushButton>
#include <QWidget>
#include <QFont>

int main( int argc, char **argv ) {
    QApplication * app = new QApplication(argc, argv);

    QWidget * box = new QWidget();
    box->resize(200, 120);

    QPushButton * quitBtn = new QPushButton("Quit", box);
    quitBtn->resize(100, 50);
    quitBtn->move(50, 35);
    quitBtn->setFont( QFont("Times", 18, QFont::Bold) );

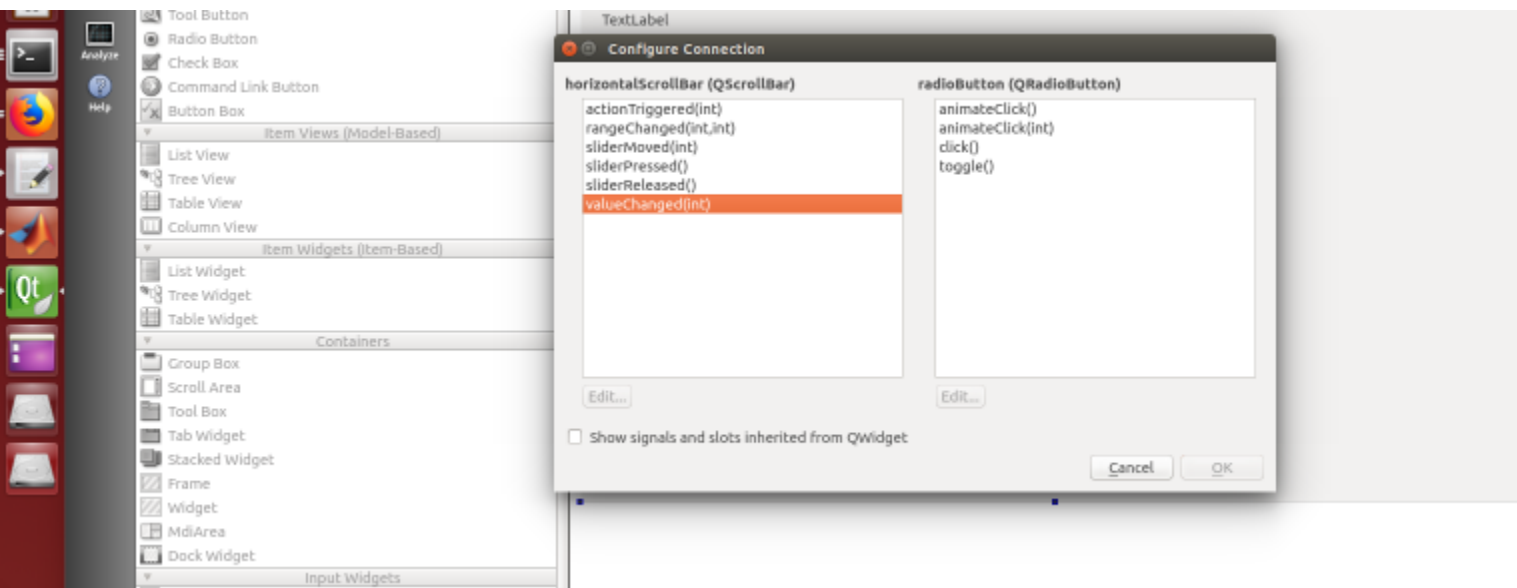
    QObject::connect( quitBtn, SIGNAL(clicked( )), app, SLOT(quit( )) ); // connexion

    box->show( );
    return app->exec( );
}
```



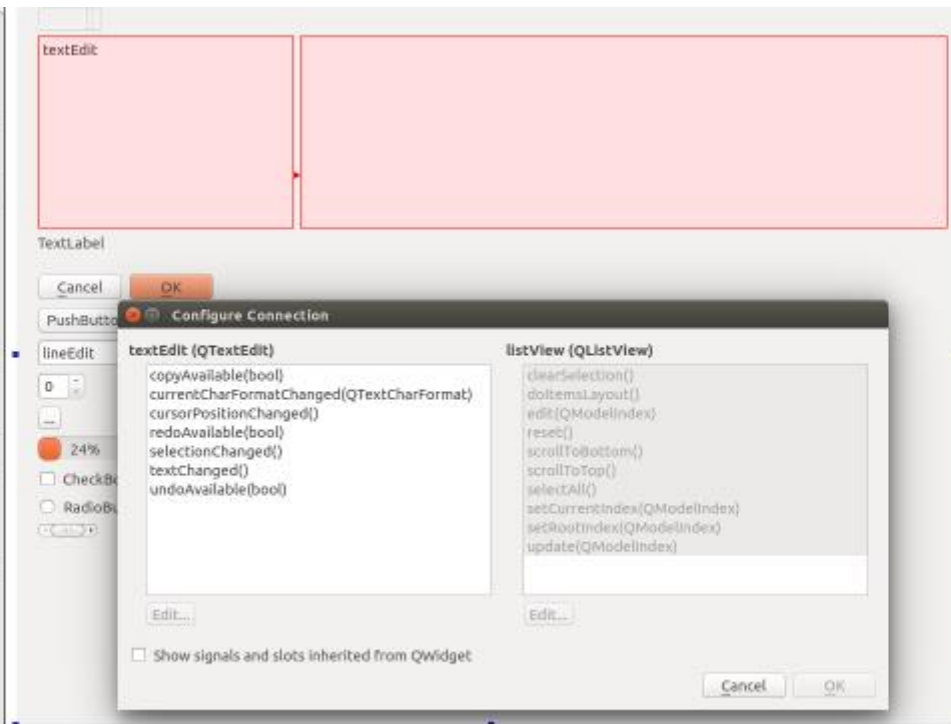
Signal/Slot

- Possibilité d'envoyer des paramètres
 - En fonction des signaux utilisés



Signal/Slot

- Possibilité d'envoyer des paramètres
 - En fonction des signaux utilisés



Signal/Slot

- Possibilité d'envoyer des paramètres
 - En fonction des signaux utilisés
 - Chaque signal a ses propres événements d'émission
 - Chaque slot a ses propres méthodes de réception
- Exemples:
 - `valueChanged(int)` ➔ dès que la valeur change, un `int` est envoyé. Il correspond à la nouvelle valeur

Signal/Slot

- Exemples:
 - Utilisation d'un spinBox et progressBar



```
QObject::connect(m_spinB, SIGNAL(valueChanged(int)), m_pBar, SLOT(setValue(int)))
```

- Pas besoin d'indiquer le nom de la variable passée en paramètre, passage uniquement du type (connexion automatique)
- Possibilités d'avoir plusieurs paramètres

Signal/Slot

- Signaux et slots prédéfinis pour chaque Widget
- ➔ Mais nombre limité de signaux et de slots
 - Possibilité de créer ses propres signaux et slots
 - Généralement, on a besoin d'ajouter des slots mais il est aussi possible de créer des signaux
 - Ajout dans une classe

Signal/Slot

- Création de signaux / slots
 - Ajouté une macro `Q_OBJECT` (cela permet d'informer le compilateur qu'il y a un nouvel élément)
➔ lancer `qmake`
 - Déclarer le signal ou le slot dans le fichier header

Public slots: retour NomMethode(paramètres)

- Ecrire la définition de la méthode dans le `.cpp`
retour NomMethode(paramètres){
 ...
}

Signal/Slot

- Création de signaux / slots
 - Il reste maintenant la connexion

```
QObject::connect(m_spinB, SIGNAL(valueChanged(int)), m_pBar,  
                SLOT(NomMethode(int)));
```

Signal/Slot

- Création de signaux / slots

Classe initiale

```
class MyFirstClass : public QWidget
{

    public:
        MyFirstClass();

    private:
        QSlider *m_slider;

};
```

Classe modifiée

```
class MyFirstClass : public QWidget
{
    Q_OBJECT

    public:
        MyFirstClass();

    public slots:
    void NomMethode(int valeur);

    private:
        QSlider *m_slider;

};
```

Signal/Slot

- Création de signaux / slots

```
void MyFirstClass::NomMethode(int valeur)
{
    //    code en fonction de votre objectif
}
```

```
QObject::connect(m_slider, SIGNAL(valueChanged(int)), this, SLOT(NomMethode(int));
```

(à placer dans votre code, en fonction de votre application)

Signal/Slot

- Création de signaux / slots
 - Pour la création d'un signal:
 - utilisation de **emit** pour émettre le signal

Événements liés à la souris

Classe QMouseEvent

contient les paramètres d'un événement lié à la souris

Appuyer ou relâcher un bouton

`void mousePressEvent(QMouseEvent*)`

Double clique

`void mouseDoubleClickEvent(QMouseEvent*)`

Déplacement de la souris

`void mouseMoveEvent(QMouseEvent*)`

Événements liés à la souris

Gestion des événements

Redéfinir la méthode associée à l'événement

```
#include « MyFirstClass »  
MyFirstClass::MyFirstClass(){  
    ...  
}  
void MyFirstClass::mousePressEvent( QMouseEvent* e){  
  
    // insérer le code de l'événement  
}
```

Événements

Exemple :

```
void MyCheckBox::mousePressEvent(QMouseEvent *event){  
    if (event->button() == Qt::LeftButton) {  
        // traiter le clique du bouton gauche de la souris ici  
    }  
    else { // propager les autres boutons à la classe de base  
        QCheckBox::mousePressEvent(event);  
    }  
}
```

Événements

D'autres types d'événements

QKeyEvent

QPaintEvent

QDragEnterEvent

...

Timer

Qtimer

signal activé après un délai fixé

Exemple

```
QTimer * timer = new QTimer ( );  
connect(timer, SIGNAL(timeout( )), widget,SLOT(doSomething( )));  
timer->start(delay);
```

...

```
timer->stop();
```

Classe QStyle

- Émulent les styles de différentes plateformes
 - Windows
 - Mac
 - ...
- Par défaut, style natif du système d'exploitation

QT Designer

QT Designer

Intégrer dans QtCreator mais peut-être utilisé séparément

Permet de « designer » directement votre interface graphique

A partir d'un fichier existant

- Nouveau fichier
- Qt → Qt Designer Form Class
- Template (forme pré-définie)

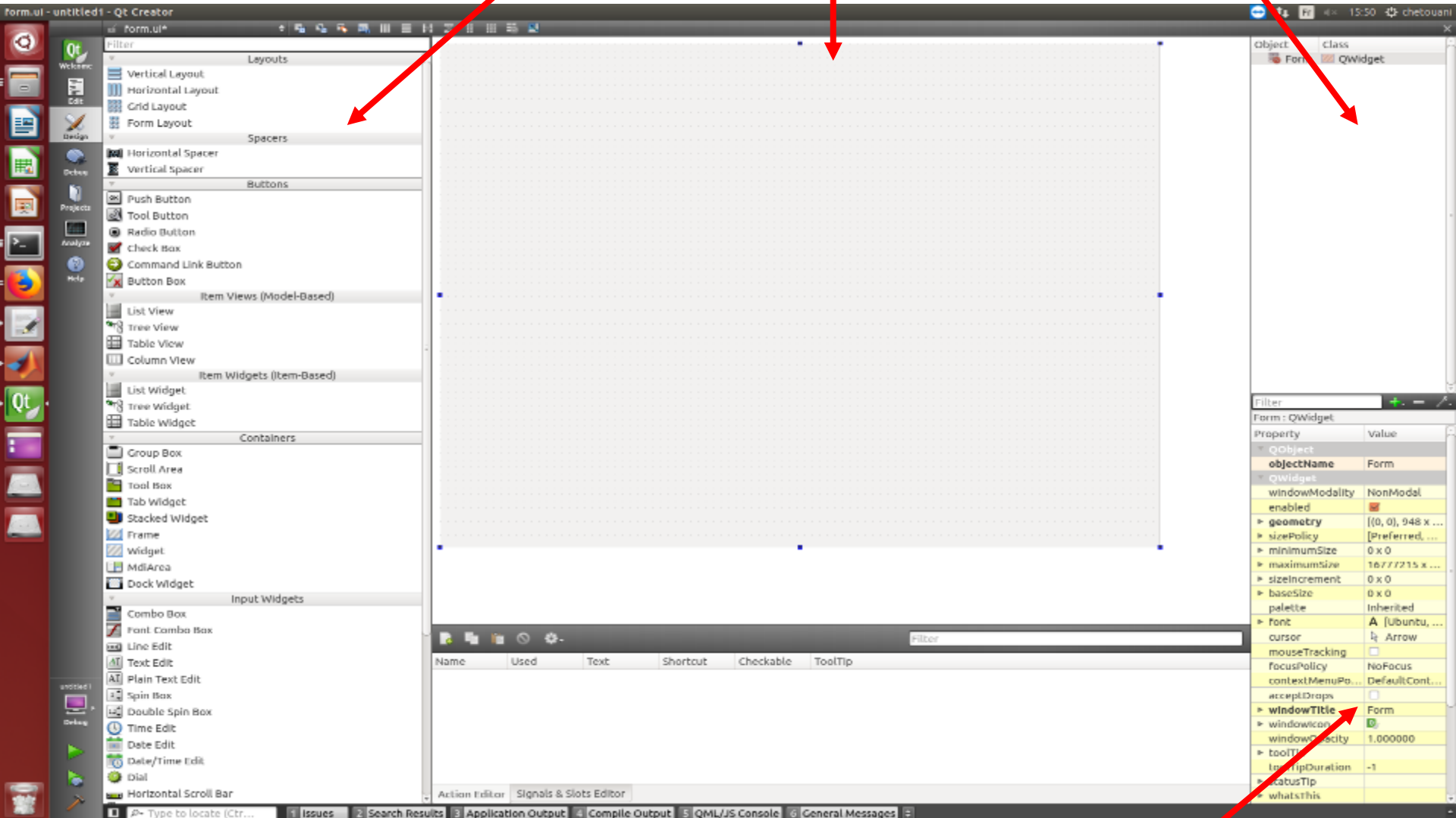
<https://openclassrooms.com/courses/programmez-avec-le-langage-c/modeliser-ses-fenetres-avec-qt-designer>

QT Designer

Contrôles disponibles

Interface

Contrôles de l'interface



Propriétés de chaque contrôle

QT Designer



Plusieurs fonctionnalités disponibles

- Editer un Widget: permet de modifier un Widget
- Editer Signaux/Slots: créer des connexions entre les signaux et les slots de vos widgets
- Editer les copains: ...
- Editer l'ordre des onglets: ...

QT Designer

Fenêtre principale

dépend du template choisi

Widgets disponibles

listing

Property Editor

propriétés du widget sélectionné
par catégorie (héritage)

Object Inspector

liste des widgets présent sur la forme

Action Editor

permet de créer des QAction

Signals & Slots Editor

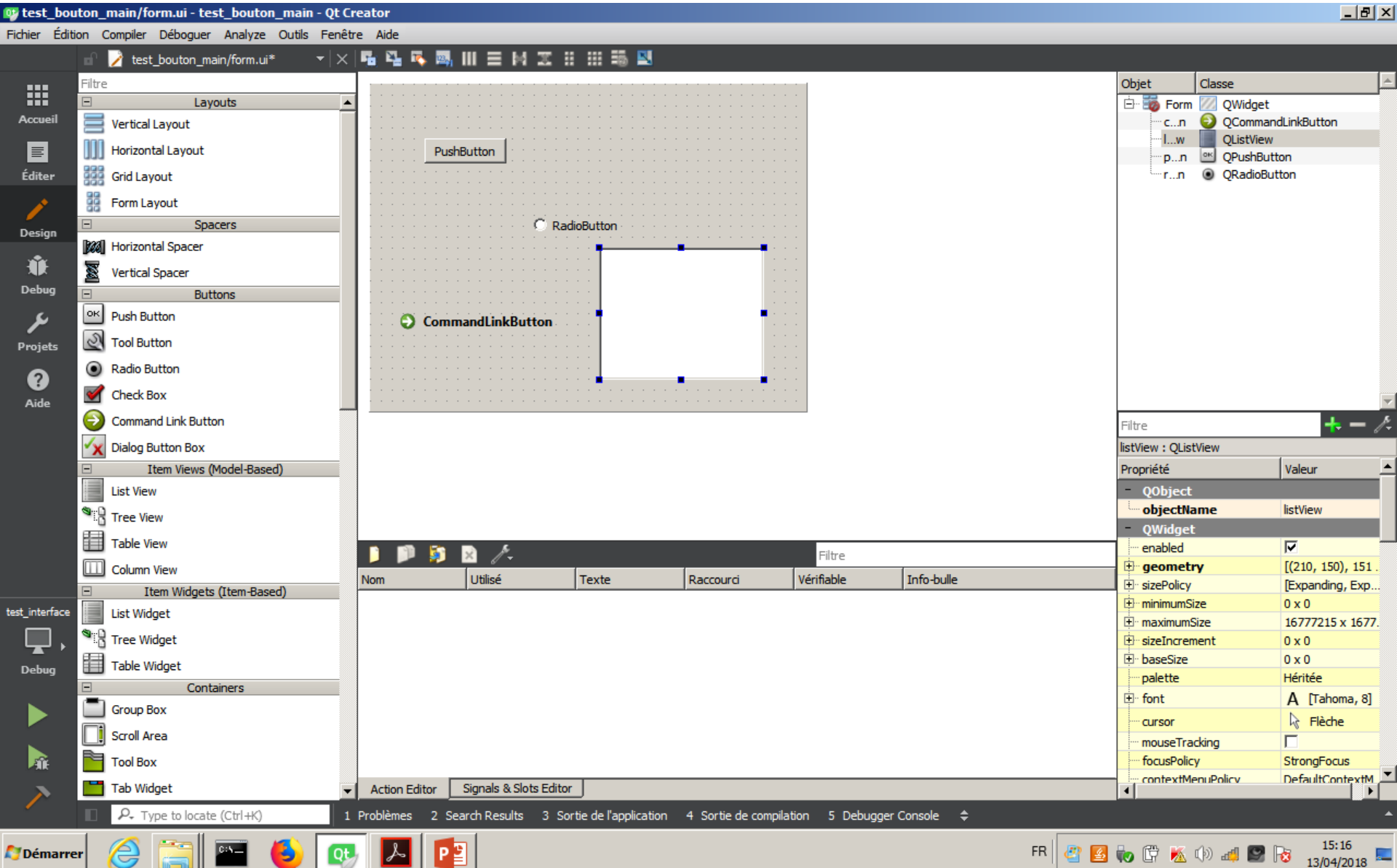
connexions signaux/slots associés au widget sélectionné

QT Designer

Créer son interface

Déplacer les éléments (similaire aux Winforms)

QT Designer



QT Designer



Layouts

mise en forme

Positionnements des widgets

- vertical

- horizontal

- grille

- Adjust size (ajustement de la taille en fonction)

- ...

Possibilité d'ajouter des layouts dans des layouts

Possibilité d'utiliser un « spacer » pour forcer l'espacement entre les widgets

QT Designer

test_bouton_main/form.ui - test_bouton_main - Qt Creator

Fichier Édition Compiler Débugger Analyze Outils Fenêtre Aide

test_bouton_main/form.ui*

Accueil Éditer Design Debug Projets Aide

Filter

Layouts

- Vertical Layout
- Horizontal Layout
- Grid Layout
- Form Layout

Spacers

- Horizontal Spacer
- Vertical Spacer

Buttons

- Push Button
- Tool Button
- Radio Button
- Check Box
- Command Link Button
- Dialog Button Box

Item Views (Model-Based)

- List View
- Tree View
- Table View
- Column View

Item Widgets (Item-Based)

- List Widget
- Tree Widget
- Table Widget

Containers

- Group Box
- Scroll Area
- Tool Box
- Tab Widget

PushButton

RadioButton

CommandLinkButton

Objet Classe

- Form QWidget
- ...n QCommandLinkButton
- ...w QListView
- ...n QPushButton
- ...n QRadioButton

Filter

Form : QWidget

Propriété	Valeur
- QObject	
objectName	Form
- QWidget	
enabled	☒
geometry	[(0, 0), 400 x 300]
sizePolicy	[Preferred, Prefe.]
minimumSize	0 x 0
maximumSize	16777215 x 1677.
sizeIncrement	0 x 0
baseSize	0 x 0
palette	Héritée
font	A [MS Shell Dlg.]
cursor	Flèche
mouseTracking	☐
focusPolicy	NoFocus
contextMenuPolicy	DefaultContextM

Nom Utilisé Texte Raccourci Vérifiable Info-bulle

Action Editor Signals & Slots Editor

Type to locate (Ctrl+K)

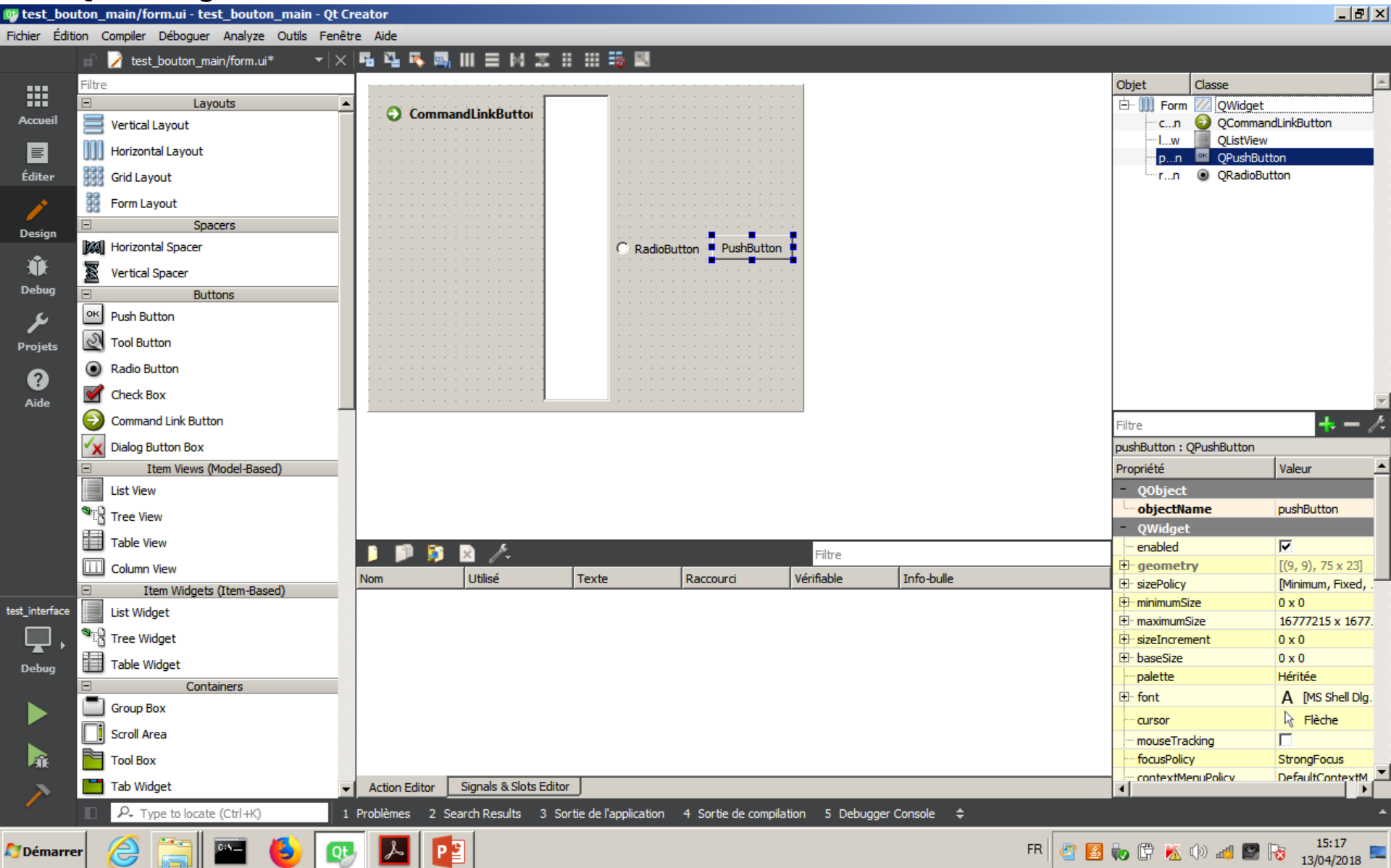
1 Problèmes 2 Search Results 3 Sortie de l'application 4 Sortie de compilation 5 Debugger Console

Démarrer

FR

15:17
13/04/2018

QT Designer



QT Designer

Edition des propriétés

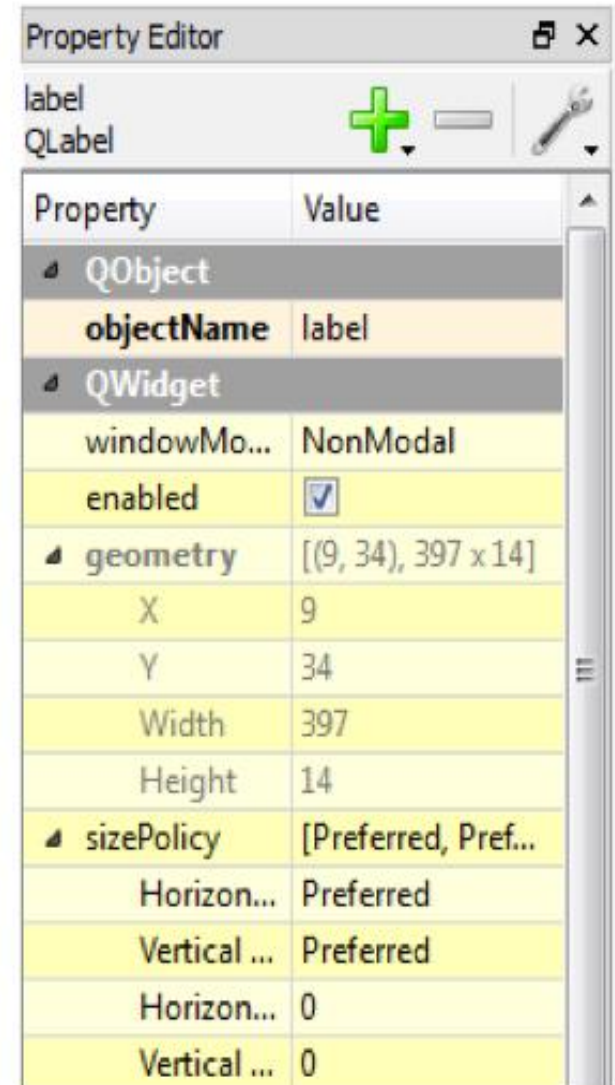
organiser en fonction de l'héritage

Exemple :

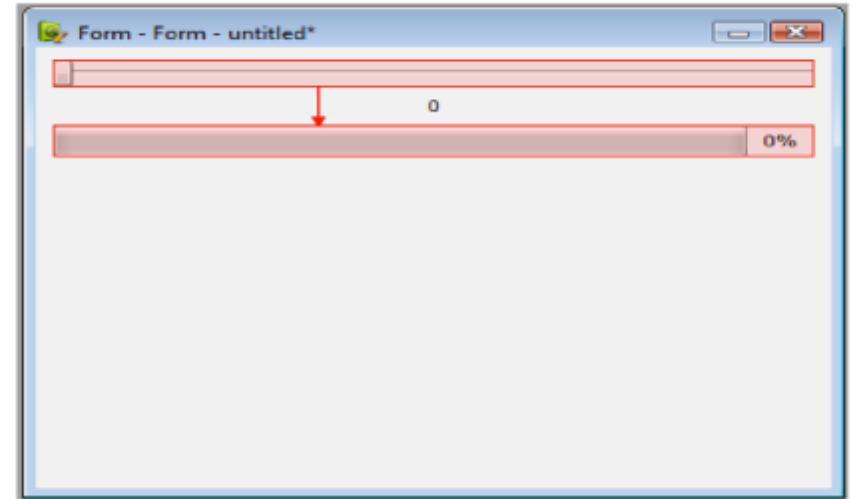
Qlabel hérite de **Qframe** qui hérite de **Qwidget** qui hérite de **Qobject**

On a donc une visualisation de propriétés en propres à la classe et celles liées aux différents héritage

Les propriétés modifiées sont en gras



QT Designer



Signaux et slots

Connexion entre les signaux et les slots

Mode Edit Signals/Slots

Sélectionner les widgets à connecter en commençant par le widget principale (celui qui va agir sur le second widget)

Ajout des signaux

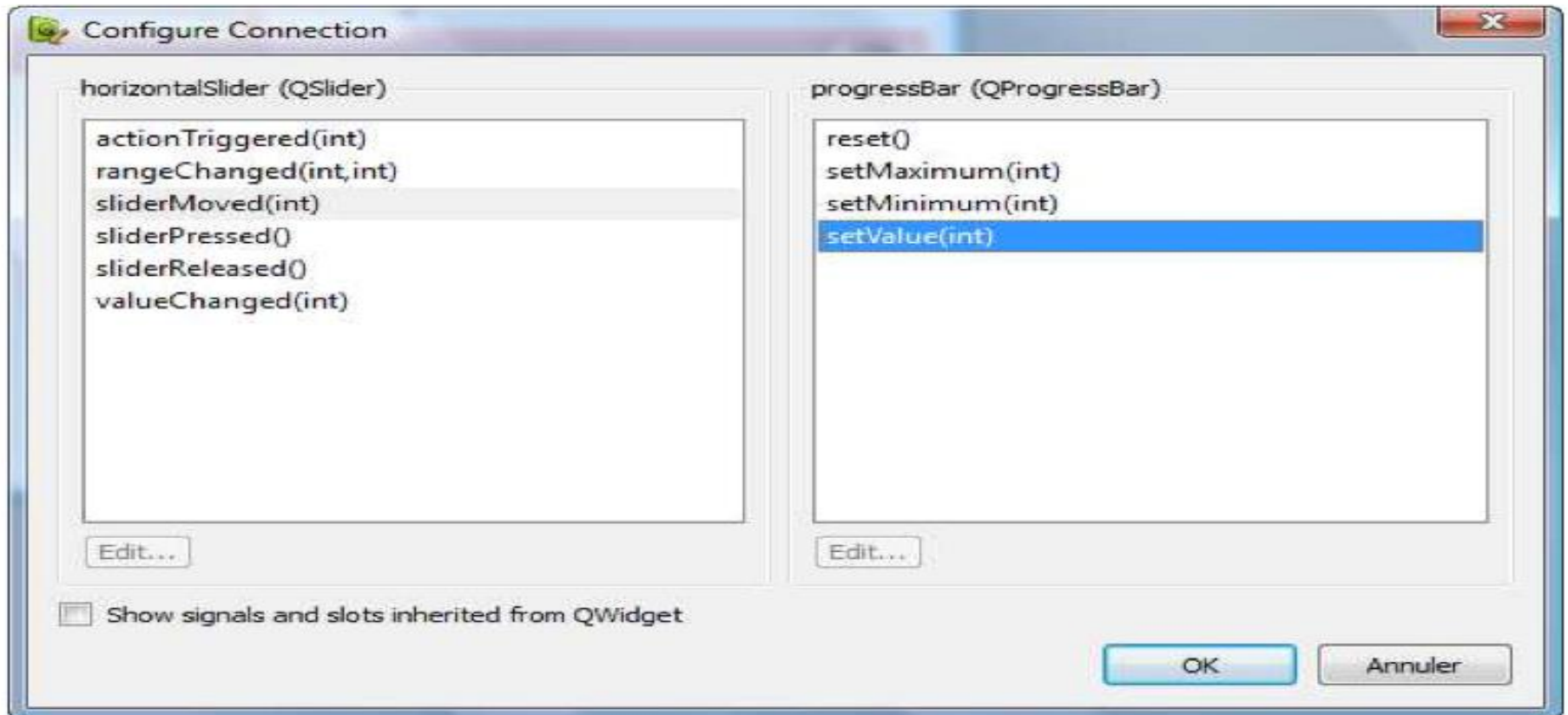
QT Designer

Sélection du signal et du slot à connecter

affichage uniquement des slots disponibles pour le signal sélectionné

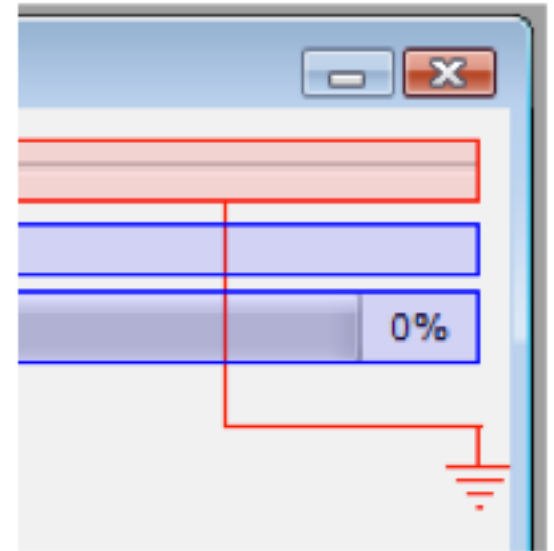
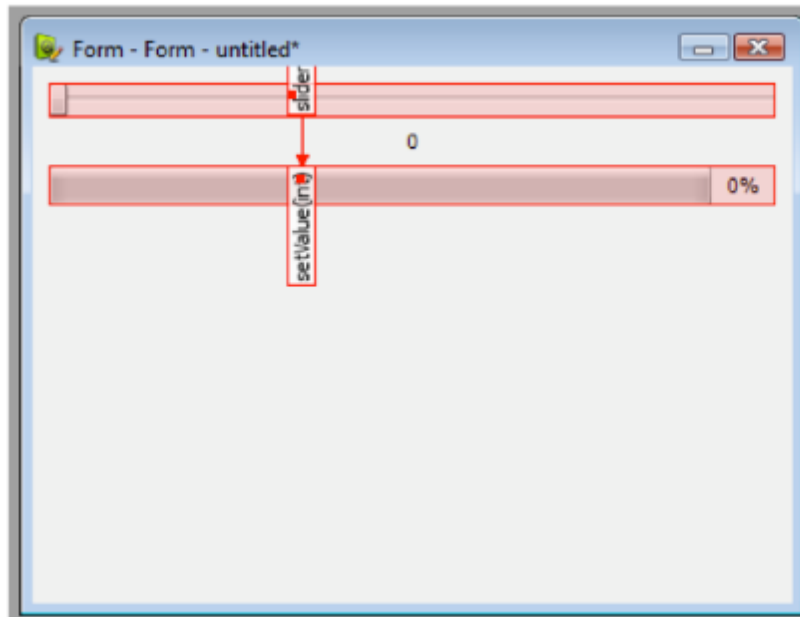
Signaux

slots



QT Designer

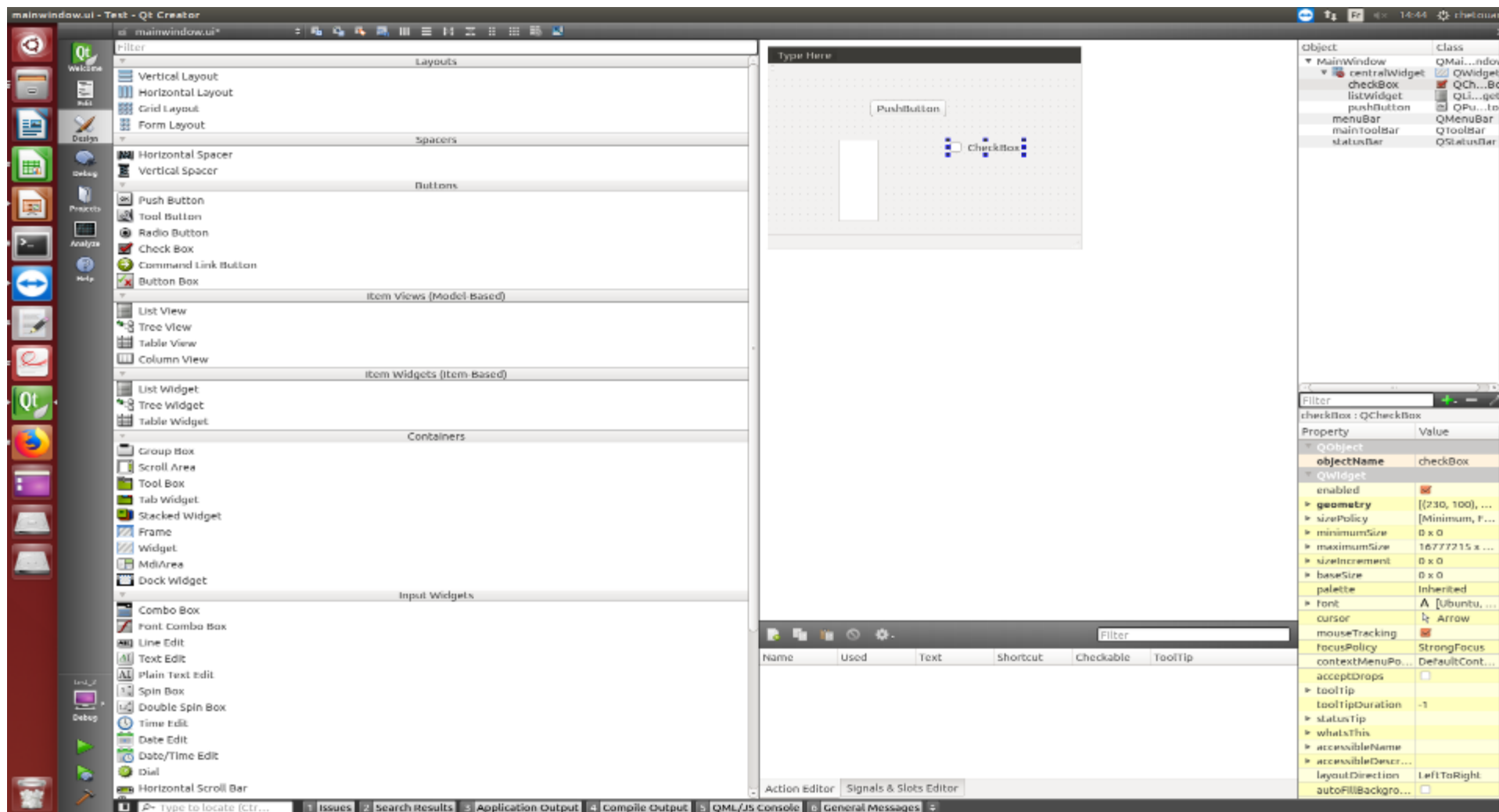
Exemple



Associe un signal à la fenêtre
(fermeture de la fenêtre)

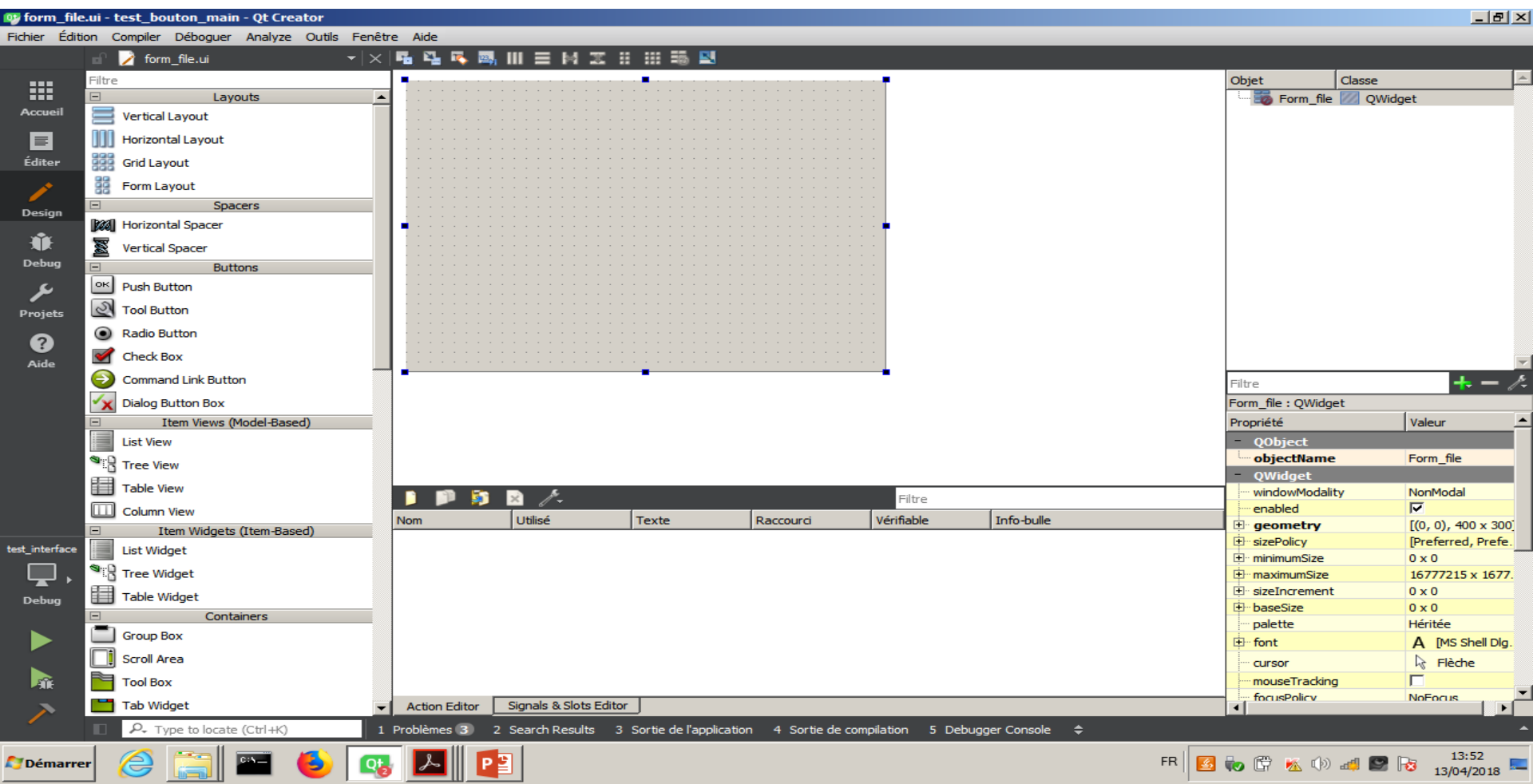
QT Designer

Utilisation de la fenêtre dans votre application



Lors de l'ajout d'une forme

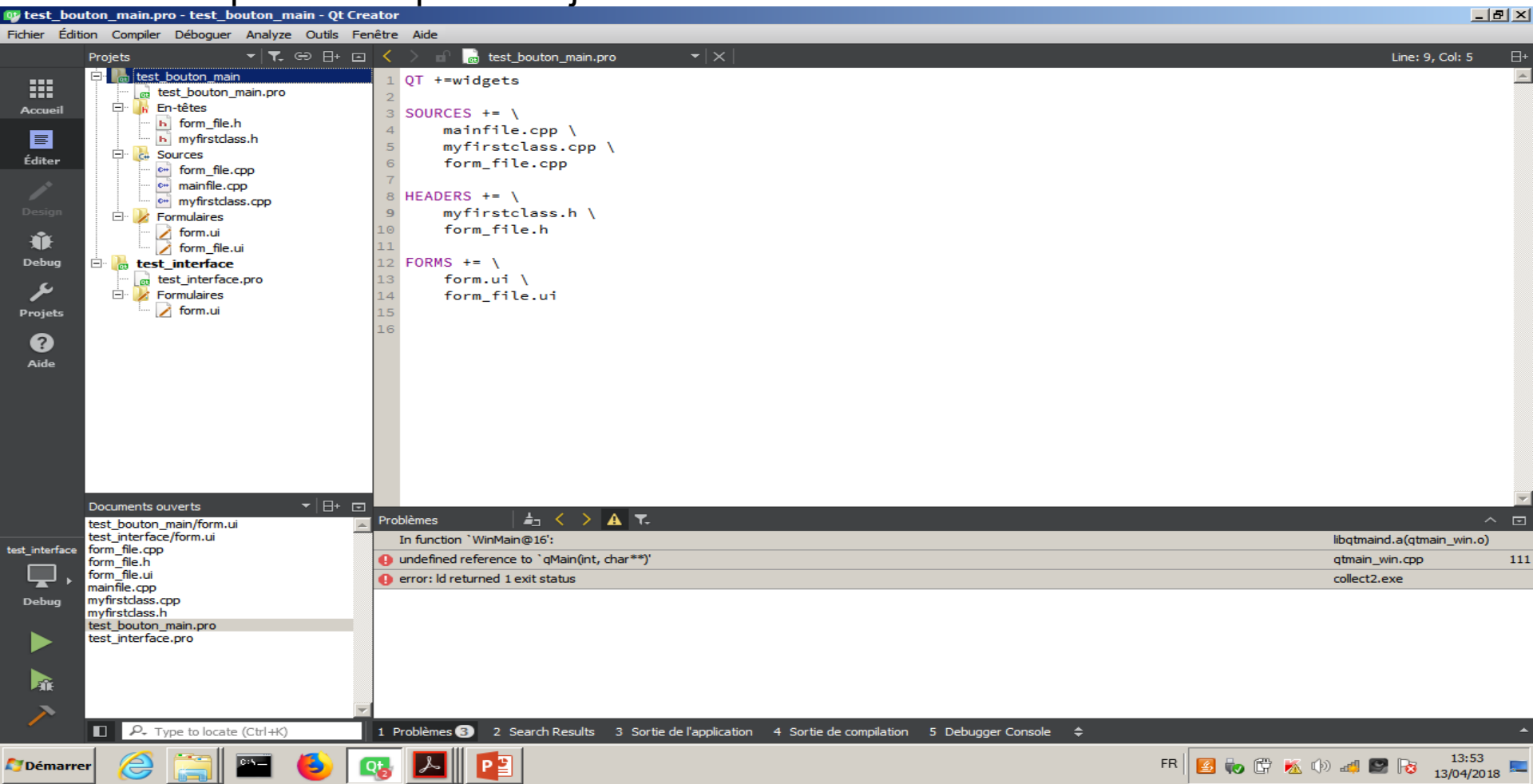
Qt Designer propose plusieurs options : ajout d'une classe associée à l'ajout d'une classe associée à l'utilisation de la fenêtre dans votre application



Qt Designer

Ajout dans le fichier .pro

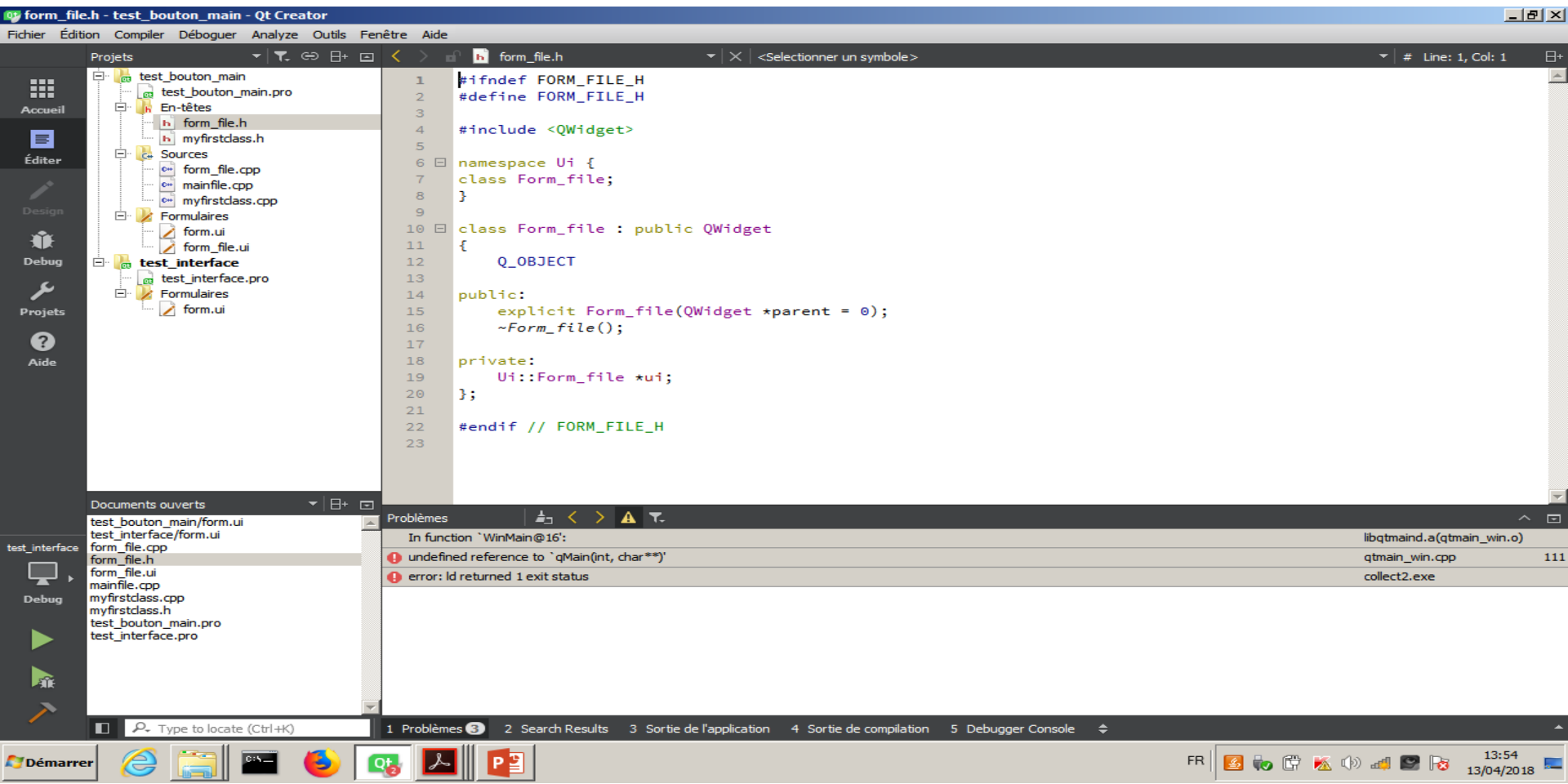
Lors de l'ajout d'une forme
plusieurs options : ajout d'une classe associée



Qt Designer

Fichier .h

Lors de l'ajout d'une forme
plusieurs options : ajout d'une classe associée



QT Designer

Fichier .h

Ligne permettant la création de votre interface

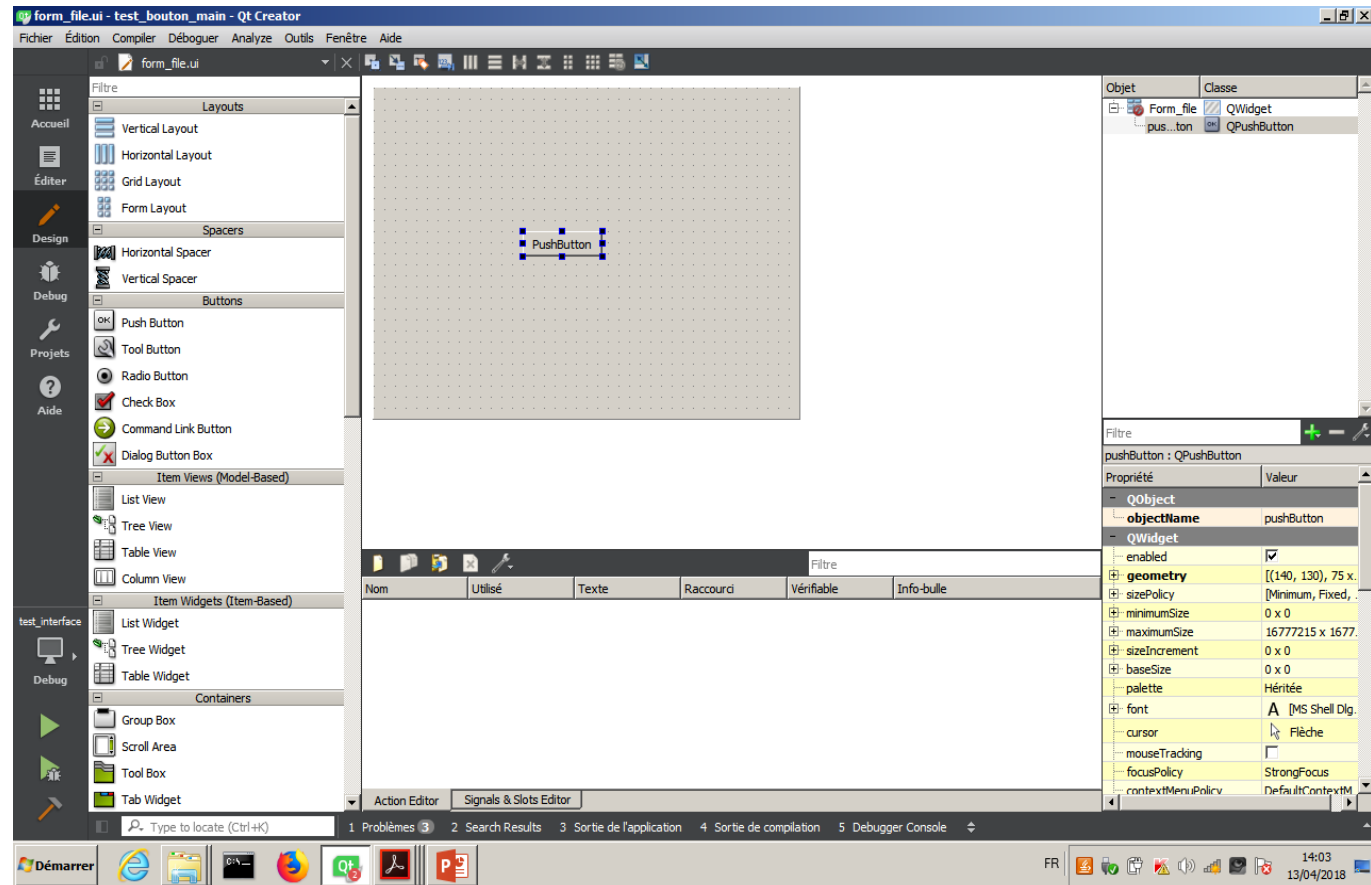
```
ui->setupUi(this);
```

Comment utiliser les éléments de la fenêtre créée ?

QT Designer

Selon votre application, accès aux éléments de votre interface via l'objet « **ui** »

Exemple QT Designer



```
#include "form_file.h" #include "ui_form_file.h"
Form_file::Form_file(QWidget *parent) : QWidget(parent), ui(new Ui::Form_file)
{
    ui->setupUi(this);
    ui->pushButton->setText("test");
}

Form_file::~Form_file() { delete ui; }
```

Qt Designer ensuite d'ajouter les connexions signaux/slots

```
connect(ui->bouton, SIGNAL(clicked()), qApp, SLOT(quit()));
```

Les interfaces créées via QT Designer propose une connexion automatique

QT Designer génère une signature spécifique

on_[Widget]_[slot]()

Exemple

Widget → BoutonTest

QT Designer Slot → clicked()

```
void on_BoutonTest_clicked()
```

QT Designer

Exemple

Widget → BoutonTest

Slot → clicked()

```
class Form_file : public QWidget {  
    Q_OBJECT  
  
public:  
    explicit Form_file(QWidget *parent = 0);  
    ~Form_file();  
  
private: Ui::Form_file *ui; };  
  
private slots:  
    void on_BoutonTest_clicked()  
  
}
```

QT Designer

Exemple

Widget → BoutonQuitter
Slot → clicked()

```
Void Form_file ::on_boutonTest_clicked()  
{  
    // Code à ajouter en fonction de votre  
    contexte  
}
```

Plus besoin de la connexion dans le constructeur

FIN